Alocação de interceptadores contra enxames mistos de drones e iscas sob informação de radar

Revisão de técnicas e avaliação comparativa reprodutível

Otávio Zordan Alves

Resumo

Ataques com dezenas a centenas de aeronaves não tripuladas baratas, lançadas em ondas e misturadas a iscas, tornaram-se frequentes na última década. Contra eles, o recurso que se esgota primeiro não é o sensor nem o efetor, e sim a capacidade de *decidir*: qual interceptador lançar contra qual ameaça, quando lançar, quando guardar e quando trocar de alvo. Essa decisão é estudada há mais de sessenta anos como problema de atribuição arma–alvo (WTA), mas a literatura recente se dividiu em comunidades que raramente se comparam entre si — pesquisa operacional, robótica multiagente, controle preditivo e aprendizado por reforço —, cada uma com seu próprio simulador e suas próprias hipóteses. Este artigo faz três coisas. Primeiro, formula o problema como um processo de decisão parcialmente observável, em que o decisor vê apenas pistas de radar ruidosas e possivelmente falsas, não distingue drones armados de iscas e paga tanto pelos impactos quanto pelos interceptadores desperdiçados. Segundo, revisa as técnicas de alocação em oito famílias, explicando a ideia, a origem e as hipóteses de cada uma. Terceiro, *implementa e compara todas as técnicas revisadas* — 23 algoritmos de alocação e 6 políticas aprendidas — num mesmo simulador aberto, com relevo real, rede de radares com difração, ruído, vieses e falsos alarmes e rastreador IMM. O conjunto de avaliação tem 192 missões em seis cidades brasileiras (São José dos Campos, Taubaté, Itajubá, São Paulo, Rio de Janeiro e Brasília): enxames de 12 a 48 vetores que misturam quatro classes de ameaça com parâmetros de fontes abertas — drones de ataque de longo alcance, iscas, munições vagantes e FPV —, atacando ativos reais de infraestrutura extraídos do OpenStreetMap, por rotas planejadas sobre o relevo e com chegada coordenada, contra uma defesa montada sem conhecimento do ataque. Duas cidades (Rio de Janeiro e Brasília) ficam reservadas para teste, o que permite medir desempenho, custo computacional e generalização com testes estatísticos pareados. [**A FAZER:** resumo dos resultados quantitativos quando os experimentos terminarem.]

Palavras-chave: atribuição arma–alvo; enxames de drones; iscas; alocação de tarefas; leilão; consenso; metaheurísticas; aprendizado por reforço; rastreamento por radar; infraestrutura crítica.

Sumário

1 Introdução **4**

1.1 O problema em uma frase 4

1.2 Origem histórica: o problema de atribuição arma–alvo 4

1.3 O problema hoje: enxames baratos que chegam em ondas 4
1.4 Por que a alocação é difícil 5
1.5 Lacunas da literatura 6
1.6 Contribuições e organização 6

2 Formulação do problema **7**
2.1 Cenário 7
2.2 O que o defensor observa 7
2.3 Formulação como processo de decisão parcialmente observável 8
2.4 Decomposição em camadas 9
2.5 Relação com o WTA clássico 10
2.6 Elementos comuns a todos os métodos 10

3 Revisão da literatura **12**
3.1 Como a revisão foi feita 12
3.2 Regras gulosas e sistemas TEWA 13
3.3 Atribuição ótima 14
3.4 WTA e retorno marginal 14
3.5 Metaheurísticas 15
3.6 Alocação distribuída 16
3.7 Alocação dinâmica 17
3.8 Alocação integrada à trajetória 17
3.9 Arquiteturas comportamentais 18
3.10 Aprendizado de máquina 18
3.11 Guiagem e rastreamento 19
3.12 Síntese 20

4 Métodos implementados **20**
4.1 Interface comum 20
4.2 Catálogo 21
4.3 Exemplo ilustrativo 21
4.4 Regras simples (M1–M5) 23
4.5 Atribuição ótima (M6–M9) 24
4.6 Alocação distribuída (M10–M12) 26
4.7 WTA: retorno marginal, solução exata e busca local (M13–M16) 27
4.8 Metaheurísticas (M17–M20) 29
4.9 Alocação dinâmica (M21 e variantes) 31
4.10 Arquiteturas comportamentais (M22, M23) 32
4.11 Políticas aprendidas (M24–M29) 33
4.12 Resumo de custo e garantias 36

5 Ambiente de simulação **36**
5.1 Radares . . . 37
5.2 Rastreador . . . 37
5.3 Ameaças . . . 38
5.4 Interceptadores . . . 38
5.5 Rodadas de decisão e fim do episódio . . . 38

6 Conjunto de missões **39**
6.1 Cidades e ativos . . . 39
6.2 Defesa . . . 40
6.3 Classes de ameaça . . . 40
6.4 Ataque . . . 40
6.5 Aleatoriedade e delineamento . . . 41
6.6 Treino e teste . . . 42

7 Protocolo experimental **43**

8 Resultados **43**
8.1 Validação das implementações . . . 43
8.2 Escolha da guiagem terminal . . . 44
8.3 Comparação dos métodos de alocação . . . 45
8.4 Efeito da composição do enxame, da cidade e da escala . . . 45
8.5 Ablações: troca de alvo, salvas, comunicação e guiagem . . . 46
8.6 Métodos aprendidos . . . 46

9 Discussão **46**
9.1 Limitações . . . 46

10 Conclusão **47**

1 Introdução

1.1 O problema em uma frase

Um conjunto de aeronaves hostis se aproxima de uma região a ser defendida. O defensor dispõe de um conjunto de *interceptadores* — pequenas aeronaves capazes de alcançar e neutralizar uma ameaça — distribuídos em bases, e de uma rede de radares que detecta as ameaças. A cada instante, o defensor precisa decidir *quais interceptadores lançar, contra quais ameaças e quando*, sabendo que cada interceptador só pode ser usado uma vez e que as ameaças mais perigosas são as que se dirigem às áreas mais importantes. Este artigo trata dessa decisão, que chamamos de *alocação de interceptadores*.

1.2 Origem histórica: o problema de atribuição arma–alvo

A pergunta "quais armas devem engajar quais alvos?" é um dos problemas mais antigos da pesquisa operacional, a disciplina que aplica modelos matemáticos a decisões de alocação de recursos. Ela foi formalizada no fim da década de 1950, no contexto da defesa contra bombardeiros e mísseis, como o problema de **atribuição arma–alvo** (*weapon–target assignment*, WTA) [1]. Na forma clássica, há um conjunto de armas e um conjunto de alvos; cada alvo tem um *valor* (quanto dano causa se sobreviver) e cada par arma–alvo tem uma *probabilidade de abate* (a chance de aquela arma destruir aquele alvo). Quer-se escolher qual arma vai contra qual alvo de modo a minimizar o valor esperado dos alvos que sobrevivem.

Um exemplo mínimo dá a intuição. Suponha dois alvos, um de valor 10 e outro de valor 1, e duas armas, cada uma com probabilidade 0,8 de destruir qualquer alvo. Mandar uma arma contra cada alvo deixa sobreviver, em média, $10 \times 0{,}2 + 1 \times 0{,}2 = 2{,}2$. Mandar as duas contra o alvo valioso deixa sobreviver $10 \times 0{,}2 \times 0{,}2 + 1 = 1{,}4$, porque o alvo de valor 10 só sobrevive se as duas armas falharem. A melhor decisão depende, portanto, dos valores e das probabilidades, e não é óbvia mesmo em instâncias pequenas.

Nos anos seguintes surgiram as primeiras heurísticas — regras práticas que dão boas soluções sem garantia de serem as melhores —, como o *máximo retorno marginal*, que designa uma arma de cada vez ao alvo em que ela mais reduz o valor sobrevivente [2]. Na década de 1980, a defesa contra mísseis balísticos motivou a versão **dinâmica** do problema, em que os engajamentos ocorrem em estágios e o defensor observa o resultado de um estágio antes de decidir o próximo [3]. Na mesma época, provou-se que o WTA é **NP-completo**: não se conhece algoritmo que o resolva exatamente em tempo que cresça só polinomialmente com o número de armas e alvos [4], o que, na prática, significa que instâncias grandes só podem ser resolvidas aproximadamente. Três revisões consolidaram a área: Cai et al. [5] sobre a versão dinâmica, Roux e van Vuuren [6] sobre sistemas de avaliação de ameaças e atribuição de armas (TEWA, na sigla em inglês) e Kline et al. [7] sobre formulações e algoritmos exatos e heurísticos.

1.3 O problema hoje: enxames baratos que chegam em ondas

Na última década, o problema mudou de escala e de natureza. Aeronaves não tripuladas baratas permitem ao atacante lançar dezenas a centenas de vetores de uma vez, e a defesa tradicional, baseada em poucos mísseis caros, deixa de ser economicamente viável: não faz sentido gastar um

míssil de alto custo contra um drone que custa uma pequena fração dele. Alguns casos ilustram a progressão:

- Em janeiro de 2018, treze pequenos drones de construção rudimentar atacaram a base aérea russa de Khmeimim e o ponto naval de Tartus, na Síria; segundo o relato russo, foram neutralizados por guerra eletrônica e defesa antiaérea [8]. Foi um dos primeiros ataques amplamente noticiados com vários drones coordenados.
- Em setembro de 2019, o ataque às instalações de petróleo de Abqaiq e Khurais, na Arábia Saudita, combinou 18 drones e cerca de sete mísseis de cruzeiro e interrompeu temporariamente cerca de metade da produção saudita, com a defesa praticamente sem engajar [9]. Mostrou que poucas dezenas de vetores baratos podem causar dano estratégico.
- Em abril de 2024, o ataque iraniano contra Israel empregou mais de 300 vetores — cerca de 170 drones, mais de 30 mísseis de cruzeiro e mais de 120 balísticos —, e, segundo as forças israelenses, 99 % foram interceptados por uma defesa multinacional coordenada [10]. Nesse caso, a coordenação de muitos efetores contra muitas ameaças foi o fator decisivo.
- Na guerra na Ucrânia, ataques noturnos com centenas de drones do tipo Shahed levaram, a partir de 2024–2025, ao uso em massa de *drones interceptadores*: aeronaves baratas lançadas contra as ameaças a partir das indicações do radar [11].

O último caso é exatamente o cenário deste artigo: muitos interceptadores baratos e *descartáveis* (cada um neutraliza uma única ameaça e é consumido), despachados a partir de um quadro de radar comum contra muitas ameaças que chegam em **ondas** — grupos sucessivos, separados por alguns minutos, que tentam **saturar** a defesa, isto é, apresentar mais ameaças simultâneas do que ela consegue tratar. Nesse regime, a qualidade da *alocação* decide quantas ameaças passam: a mesma frota de interceptadores, mal distribuída, deixa passar ameaças que uma distribuição melhor teria detido.

1.4 Por que a alocação é difícil

Três fatores tornam a decisão difícil.

1. **Explosão combinatória.** Com 48 ameaças e 60 interceptadores, o número de maneiras de designar interceptadores a ameaças é astronômico (maior que 10^{60}), e a decisão precisa ser tomada em fração de segundo e refeita continuamente.
2. **Informação imperfeita.** O defensor não vê as aeronaves; vê **pistas de radar**. Uma pista é a estimativa, produzida por um algoritmo de rastreamento a partir das medidas dos radares, da posição e da velocidade de algo que se move. Essa estimativa tem erro; não traz a identidade da aeronave; desaparece quando a aeronave se esconde atrás do relevo; e pode ser um **falso alarme** — uma pista que não corresponde a aeronave nenhuma, criada por ruído ou por reflexos do terreno. Nos nossos cenários, a maioria das pistas pode ser falsa.
3. **Ameaças heterogêneas e iscas.** Um ataque real mistura vetores muito diferentes: drones de ataque de longo alcance, rápidos e de rota pré-programada; munições vagantes, que orbitam o alvo antes de mergulhar; drones FPV, lentos, rentes ao solo e pilotados,

capazes de manobrar; e **iscas**, aeronaves baratas sem carga explosiva, com assinatura de radar parecida, lançadas para atrair interceptadores. Parte das ameaças voa baixo para se esconder do radar no relevo. Quase todos os métodos de alocação estimam onde a ameaça estará supondo que ela mantém a velocidade atual, e nenhum tem como saber se uma pista é um drone armado ou uma isca — e cada interceptador gasto numa isca é um a menos para a próxima ameaça real.

1.5 Lacunas da literatura

A literatura recente oferece muitas propostas, mas três lacunas impedem saber qual usar:

1. **Hipóteses otimistas.** A maior parte da literatura de WTA supõe alvos conhecidos e probabilidades de abate dadas; trabalhos de robótica e controle supõem consciência situacional perfeita e ameaças em linha reta ou com velocidade constante. Nenhuma dessas hipóteses vale contra um enxame misto, com iscas, visto por radar.

2. **Escala pequena e cenários abstratos.** Os trabalhos brasileiros mais próximos, do Instituto Tecnológico de Aeronáutica [12–15], modelam com cuidado a dinâmica dos defensores, mas usam de 3 a 6 ameaças. Em geral, os cenários da literatura são sintéticos: alvos e rotas sorteados, e radares posicionados sabendo de onde o ataque virá.

3. **Comparações incomparáveis.** Os trabalhos de aprendizado por reforço [16–18] e de contra-enxame [19] usam simuladores, métricas e linhas de base próprios — muitas vezes uma só linha de base. Não é possível saber, a partir da literatura, se uma política aprendida supera o método húngaro, ou se uma árvore de comportamento supera um leilão.

1.6 Contribuições e organização

Este artigo contribui com:

- uma **formulação** do problema como processo de decisão parcialmente observável, decomposta em avaliação de ameaças, alocação e guiagem, que explicita o que cada família de métodos supõe sobre a informação e sobre o alvo (Seção 2);
- uma **revisão** das técnicas de alocação em oito famílias, da pesquisa operacional ao aprendizado por reforço, explicando a ideia de cada uma e comparando os trabalhos mais próximos (Seção 3);
- a **implementação de todas as técnicas revisadas** numa interface comum — 29 métodos, cada um descrito por origem, funcionamento, adaptações e parâmetros —, com a verificação de que as implementações reproduzem as equivalências teóricas conhecidas (Seção 4);
- um **conjunto de missões realistas** e aberto: 192 missões em seis cidades brasileiras, com enxames de até 48 vetores que misturam drones de ataque de longo alcance, iscas, munições vagantes e FPV (parâmetros de fontes abertas), atacando ativos reais de infraestrutura do OpenStreetMap por rotas planejadas sobre o relevo, contra uma defesa montada sem conhecer o ataque (Seção 6);
- uma **avaliação reprodutível** num simulador aberto (Seção 5), com divisão treino/teste por

cidade, sementes pareadas e testes estatísticos, que mede tanto o dano quanto o desperdício de interceptadores (Seções 7 e 8).

A Seção 9 discute os resultados, as limitações e as recomendações.

2 Formulação do problema

Esta seção descreve o cenário, escreve o problema de decisão em linguagem matemática, explica por que ele não pode ser resolvido exatamente, mostra como os métodos práticos o decompõem e define as grandezas que quase todos os métodos usam.

2.1 Cenário

Uma região contém **zonas prioritárias** $\mathcal{Z}$ — círculos que representam ativos importantes, como uma base, uma usina ou uma subestação — e **bases de interceptadores**, de onde partem os defensores. Um conjunto $\mathcal{A}$ de N ameaças entra na região em ondas e segue rotas que terminam num **ponto de impacto**, onde a ameaça mergulha contra o alvo. Cada ameaça pertence a uma **classe** — drone de ataque de longo alcance, munição vagante, drone FPV ou **isca** — que determina a sua velocidade, altura, manobra e o **valor de dano** ν_a da sua carga: $\nu_a = 1$, 0,5 e 0,25 para as três classes armadas e $\nu_a = 0$ para a isca, que não carrega explosivo e existe só para atrair interceptadores. Se uma ameaça chega ao impacto sem ser interceptada, causa um dano $w_a = \nu_a\, w_P$, com $w_P = 10$, se o impacto é dentro de uma zona prioritária, e $w_a = \nu_a\, w_O$, com $w_O = 1$, fora dela; a razão de 10 para 1 expressa que proteger os ativos defendidos é muito mais valioso que evitar impactos em ativos secundários.

O defensor tem $M \geq N$ interceptadores. Cada um voa a velocidade v_d, tem autonomia T_d (tempo máximo de voo) e um **sensor próprio** de curto alcance r_s: enquanto está longe da ameaça, o interceptador depende das pistas de radar para saber para onde ir; quando chega a menos de r_s, passa a ver a ameaça diretamente. Um interceptador neutraliza a ameaça quando passa a menos de r_c dela (**contato**) e é consumido nesse contato. Um interceptador que é lançado e não neutraliza uma ameaça armada — porque esgota a autonomia sem contato, porque abate uma isca ou porque ainda voa sem uso quando o ataque termina — é **desperdiçado**: foi gasto sem proteger nada. A Tabela 1 resume a notação.

2.2 O que o defensor observa

O defensor não vê as aeronaves. A cada instante ele recebe o **quadro de radar** $\mathcal{P}(t)$, a lista de pistas mantida pelo rastreador. Cada pista p_j traz uma posição estimada $\boldsymbol{x}_j$, uma velocidade estimada $\boldsymbol{v}_j$, uma medida da incerteza dessas estimativas, a idade (há quanto tempo foi atualizada) e o número de medidas que a sustentam. Três coisas *não* são observadas e precisam ser inferidas ou ignoradas: (i) qual aeronave uma pista representa, ou se representa alguma (pode ser falso alarme); (ii) o ponto de impacto e, portanto, se a ameaça vai cair numa zona prioritária; e (iii) a classe da ameaça — em particular, se é uma isca, que voa como as demais e tem assinatura de radar parecida. Além disso, ameaças escondidas pelo relevo simplesmente não aparecem no quadro.

Tabela 1. Notação principal.

Símbolo	Significado
$\mathcal{A}$, N; $\mathcal{D}$, M	aeronaves e seu número; interceptadores e seu número
ν_a; w_a; $w_P = 10$, $w_O = 1$	valor de dano da classe (isca: 0); dano da ameaça a; peso dentro e fora de zona prioritária
W, $\eta = 0{,}2$	interceptadores desperdiçados num episódio; penalidade por desperdício
v_d, T_d, r_s, r_c	velocidade, autonomia, alcance do sensor e raio de contato do interceptador
$\mathcal{P}(t) = \{p_j\}$	quadro de radar: pistas com posição $\boldsymbol{x}_j$, velocidade $\boldsymbol{v}_j$, incerteza, idade
$\mathcal{D}_t$	fila de interceptadores que decidem na rodada t
$x_{ij} \in \{0, 1\}$	designação do interceptador i à pista j (1 se designado)
$t^{\star}_{ij}$, c_{ij}	tempo previsto de interceptação; custo com histerese (E2)
w_j, t^{Z}_{j}	peso de prioridade e tempo até a zona inferidos da pista (E3)
q_{ij}, V_j, S_j	probabilidade de sucesso, valor e sobrevivência da pista (E4)
$F(\boldsymbol{x})$	valor esperado destruído numa rodada (10)
D, C	dano normalizado (2) e custo (3) de um episódio

2.3 Formulação como processo de decisão parcialmente observável

Um **processo de decisão de Markov** (MDP) descreve um sistema que evolui no tempo sob as ações de um decisor: em cada instante o sistema está num *estado*; o decisor escolhe uma *ação*; o sistema passa a um novo estado de forma possivelmente aleatória (a *transição*), e isso gera um custo ou uma recompensa. "De Markov" significa que o próximo estado depende só do estado e da ação atuais, e não de todo o passado. Quando o decisor não vê o estado, mas apenas uma *observação* parcial e ruidosa dele, o processo é **parcialmente observável** (POMDP). Nesse caso, a melhor decisão depende de todo o histórico de observações, porque é desse histórico que o decisor infere o estado. Uma **política** é a regra que associa a cada histórico uma ação, e resolver o problema é encontrar a política de menor custo esperado.

O nosso problema é um POMDP com decisões a cada $\Delta_d = 2$ s (as **rodadas de decisão**):

- **Estado** s_t: posição, velocidade, rota, classe e situação (viva, interceptada ou caída) de cada ameaça; posição, autonomia restante, alvo e situação (na base, em voo ou consumido) de cada interceptador; e o estado interno do rastreador.
- **Observação** o_t: o quadro de radar $\mathcal{P}(t)$, o estado dos próprios interceptadores e as zonas prioritárias.
- **Ação** a_t: em cada rodada decide a **fila** $\mathcal{D}_t$, formada pelos interceptadores ociosos nas bases e pelos interceptadores em voo cuja reavaliação venceu. Cada um recebe uma pista ou a *ação nula* (esperar na base ou manter o alvo atual). A ação é uma matriz binária $\boldsymbol{x}$, com $x_{ij} = 1$ se o interceptador i é designado à pista j, sujeita a

$$\sum_{j} x_{ij} \le 1 \quad \forall i, \qquad \sum_{i} x_{ij} + n^{0}_{j} \le \bar{n} \quad \forall j. \tag{1}$$

  A primeira restrição diz que cada interceptador persegue no máximo uma pista; a segunda, que cada pista recebe no máximo $\bar{n}$ interceptadores, contando os n^{0}_{j} que já a perseguem ($\bar{n} = 1$ por padrão).

- **Transição**: as ameaças avançam pelas rotas (as FPV manobram e algumas reagem ao

interceptador); os interceptadores em trânsito voam em direção ao ponto previsto de encontro com a sua pista e, ao entrar no alcance do sensor, passam a perseguir a aeronave diretamente; os radares medem e o rastreador atualiza as pistas. Há contato quando um interceptador passa a menos de r_c de uma aeronave, e impacto quando uma aeronave chega ao fim da rota.

- **Objetivo**: minimizar o valor esperado do **dano normalizado**,

$$\min_{\pi} \ \mathbb{E}_{\pi}[D], \qquad D = \frac{\sum_{a \in \mathcal{A}} w_a \, \mathbf{1}\{a \text{ atinge o impacto}\}}{\sum_{a \in \mathcal{A}} w_a} \in [0,1], \tag{2}$$

em que $\mathbf{1}\{\cdot\}$ vale 1 se a condição é verdadeira e 0 caso contrário. D é a fração do dano possível que de fato ocorreu: vale 0 se todas as ameaças armadas foram interceptadas e 1 se nenhuma foi. A normalização permite comparar cenários com números diferentes de ameaças. As iscas não entram em (2), porque $w_a = 0$, mas pesam no **custo**, que soma ao dano a penalidade dos interceptadores desperdiçados:

$$C = \frac{\sum_{a \in \mathcal{A}} w_a \, \mathbf{1}\{a \text{ atinge o impacto}\} + \eta \, W}{\sum_{a \in \mathcal{A}} w_a}, \qquad \eta = 0{,}2. \tag{3}$$

Um interceptador desperdiçado custa um quinto do impacto de uma ameaça plena num ativo não defendido. Com $\eta = 0$, $C = D$; com $\eta > 0$, um método que dispara contra tudo o que aparece — e, portanto, contra as iscas — é penalizado mesmo que não deixe passar nada. Reportamos as duas grandezas: D mede a proteção, e C, a proteção descontado o gasto.

Por que não resolver o POMDP diretamente. Resolver exatamente um POMDP exige raciocinar sobre a distribuição de probabilidade de todos os estados possíveis, o que só é viável em problemas minúsculos. Aqui, o estado é contínuo e de alta dimensão (posições e velocidades de dezenas de aeronaves e interceptadores), o espaço de ações é combinatório (as designações possíveis entre dezenas de interceptadores e pistas) e a classe e o comportamento de cada ameaça não são conhecidos pelo defensor. Todos os métodos práticos, inclusive os aprendidos, recorrem a aproximações: decompõem o problema e resolvem cada parte com informação local.

2.4 Decomposição em camadas

A decisão completa se decompõe naturalmente em três camadas, que correspondem às comunidades da literatura:

1. **Avaliação de ameaças**: decidir quais pistas são aeronaves reais, quão perigosas são e quanto tempo há até que causem dano.
2. **Alocação**: dada essa avaliação, decidir qual interceptador vai contra qual pista nesta rodada, quais ficam esperando e quais trocam de alvo.
3. **Guiagem**: dado o alvo, levar o interceptador até ele.

Este artigo compara técnicas da camada de *alocação*. As outras duas são mantidas fixas e iguais

para todos os métodos: a avaliação de ameaças é uma regra física comum, descrita abaixo, ou um classificador aprendido (nas redes neurais); a guiagem é uma lei fixa, escolhida por um experimento separado (Seção 8.2). Fixar as outras camadas garante que as diferenças medidas venham da alocação.

2.5 Relação com o WTA clássico

Se considerarmos uma única rodada, a alocação é uma instância do WTA estático [1]: designar armas i a alvos j de valor V_j, com probabilidade de abate q_{ij}, minimizando o valor esperado que sobrevive,

$$\min_{x} \sum_{j} V_j \prod_{i} (1 - q_{ij})^{x_{ij}} \quad \text{s.a. (1).} \tag{4}$$

A leitura é direta: $(1 - q_{ij})$ é a chance de a arma i falhar contra o alvo j; o expoente x_{ij} faz o fator aparecer só se a arma foi designada; o produto sobre i é, portanto, a chance de *todas* as armas designadas a j falharem, isto é, de j sobreviver; multiplicada pelo valor V_j e somada sobre os alvos, dá o valor sobrevivente esperado. O produto é o que torna o problema não linear e difícil.

O nosso problema difere do WTA clássico em três pontos. É *dinâmico*: a alocação se repete a cada rodada e pode ser revista, como no WTA dinâmico [3]. É *parcialmente observável*: alocam-se pistas, que podem ser falsas ou estar erradas, e não alvos conhecidos. E é *acoplado à cinemática*: a probabilidade de sucesso q_{ij} não é dada, e depende de quanto tempo o interceptador leva para chegar e de como o alvo manobra. No caso particular em que cada alvo recebe no máximo um interceptador e todas as probabilidades são próximas de 1, o produto em (4) desaparece, e o problema se torna um *problema de atribuição linear*, que pode ser resolvido exatamente e rapidamente [20] (Seção 4.5).

2.6 Elementos comuns a todos os métodos

Quatro grandezas, calculadas apenas a partir das observações, são usadas por quase todos os métodos. Definimo-las aqui uma única vez; os métodos da Seção 4 se referem a elas como (E1) a (E4).

(E1) Filtro de plausibilidade. Se cada pista recebesse um interceptador, os falsos alarmes consumiriam boa parte da frota. A forma mais simples de *avaliação de ameaças* é uma regra física: as classes de ameaça conhecidas voam dentro de um envelope de velocidade, razão de subida e altura; uma pista fora desse envelope provavelmente é falsa. O envelope precisa cobrir todas as classes, inclusive o mergulho final, e por isso é largo. Os métodos só consideram pistas com

$$\Pi(p_j) = \mathbf{1}\{12 \le \|\boldsymbol{v}_j^h\| \le 75\}\ \mathbf{1}\{|v_j^z| \le 60\}\ \mathbf{1}\{-50 \le h_j^{\mathrm{AGL}} \le 2500\} = 1, \tag{5}$$

em que $\boldsymbol{v}_j^h$ é a velocidade horizontal estimada (m/s), v_j^z a vertical (m/s) e h_j^{AGL} a altura sobre o terreno (m). Os falsos alarmes do modelo de radar tendem a ter velocidades incoerentes, porque resultam de associações espúrias entre medidas; uma aeronave que manobra forte pode, por sua vez, ser momentaneamente rejeitada, que é o custo dessa regra. Note que o filtro separa

aeronaves de falsos alarmes, mas não separa iscas de drones armados: uma isca é uma aeronave e passa no filtro. As redes neurais trocam (5) por um classificador aprendido.

(E2) Custo de interceptação e histerese. A grandeza física central da alocação é o tempo que um interceptador leva para alcançar uma pista. Se o interceptador i está em $\boldsymbol{x}_i$ e a pista j está em $\boldsymbol{x}_j$ com velocidade $\boldsymbol{v}_j$, e supondo que a pista mantém essa velocidade, o encontro ocorre no instante t em que a distância percorrida pelo interceptador, $v_d\,t$, iguala a distância até a posição futura da pista, $\|\boldsymbol{x}_j + \boldsymbol{v}_j t - \boldsymbol{x}_i\|$. Elevando ao quadrado, t é a menor raiz positiva de

$$(\|\boldsymbol{v}_j\|^2 - v_d^2)\,t^2 + 2\,(\boldsymbol{r}_{ij}\cdot\boldsymbol{v}_j)\,t + \|\boldsymbol{r}_{ij}\|^2 = 0, \qquad \boldsymbol{r}_{ij} = \boldsymbol{x}_j - \boldsymbol{x}_i, \tag{6}$$

que chamamos de **tempo previsto de interceptação** $t^\star_{ij}$. Essa previsão supõe *velocidade constante*: é exatamente a hipótese que uma manobra evasiva viola, e o erro aparece como pistas que "fogem" do ponto previsto. O custo usado pelos métodos é

$$c_{ij} = \begin{cases} \kappa^{\mathbf{1}\{j=j_i\}}\, t^\star_{ij}, & t^\star_{ij} \le T_d - \tau_i, \\ \infty, & \text{caso contrário,} \end{cases} \tag{7}$$

em que τ_i é o tempo que o interceptador já voou: se o encontro exigir mais autonomia do que resta, o par é inviável (custo infinito). O fator $\kappa = 0{,}7$ é uma **histerese**, termo que designa a tendência de um sistema de permanecer no estado em que está. Para um interceptador já em voo, a sua pista atual j_i entra com custo multiplicado por 0,7, como se estivesse 30 % mais perto. Assim, ele só troca de alvo se a alternativa for claramente melhor. Sem histerese, duas pistas a distâncias parecidas, cujas estimativas oscilam com o ruído do radar, fariam o interceptador trocar de alvo a cada rodada e não chegar a nenhum. Por exemplo, se a pista atual está a 50 s e outra a 40 s, o custo atual é $0{,}7 \times 50 = 35 < 40$, e o interceptador mantém o alvo; se a outra estiver a 30 s, ele troca.

(E3) Peso de prioridade e tempo até a zona. O defensor não sabe onde cada aeronave vai cair, mas pode inferir a prioridade pela geometria da pista em relação às zonas. Sejam d_j a distância da pista à borda da zona prioritária mais próxima (negativa se já está dentro) e $\dot{d}_j$ a velocidade com que ela se aproxima do centro dessa zona. Definimos o **peso de prioridade** e o **tempo até a zona**

$$w_j = \begin{cases} w_P, & d_j \le 0, \\ (w_P + w_O)/2, & \dot{d}_j > 5 \text{ m/s e } d_j < 3 \text{ km}, \\ w_O, & \text{caso contrário;} \end{cases} \qquad t^Z_j = \begin{cases} 0, & d_j \le 0, \\ d_j/\dot{d}_j, & \dot{d}_j > 1 \text{ m/s}, \\ \infty, & \text{caso contrário.} \end{cases} \tag{8}$$

Uma pista dentro de uma zona recebe o peso máximo; uma pista fora, mas que se aproxima depressa de uma zona próxima, recebe um peso intermediário; as demais, o peso mínimo. O tempo até a zona é a distância dividida pela velocidade de aproximação, e é infinito para pistas que não se aproximam. Rotas planejadas sobre o relevo, que passam perto de ativos que não são

o seu alvo (Seção 6), enganam exatamente essa inferência.

(E4) Modelo de sucesso e objetivo de valor esperado. Os métodos derivados do WTA precisam de uma probabilidade de sucesso q_{ij}, que o problema não fornece. Estimamo-la por

$$q_{ij} = P_T\, e^{-c_{ij}/\tau_q}\, \mathbf{1}\{c_{ij} < \infty\}, \qquad P_T = 0{,}85,\ \tau_q = 600 \text{ s}. \tag{9}$$

O fator P_T é a chance de contato quando o interceptador chega perto do alvo, medida no experimento de guiagem (Seção 8.2). O fator exponencial reduz essa chance para trânsitos longos, porque quanto mais tempo o interceptador voa guiado pelo radar, mais a pista envelhece, mais o alvo pode manobrar ou sair da cobertura. Com esses valores, um encontro a 30 s tem $q \approx 0{,}81$, e um a 300 s tem $q \approx 0{,}52$.

Cada pista tem valor $V_j = w_j$. Se já há interceptadores em voo contra a pista j (o conjunto $\mathcal{F}_j$, que não decide nesta rodada), a chance de ela ainda sobreviver é $S_j = \prod_{k\in\mathcal{F}_j}(1 - q_{kj})$, a **sobrevivência atual**. O objetivo de uma rodada é o **valor esperado destruído**,

$$F(\boldsymbol{x}) = \underbrace{\sum_j V_j S_j \Big[1 - \prod_i (1 - q_{ij})^{x_{ij}}\Big]}_{\text{valor esperado destruído}} - \underbrace{\sum_i \ell_i \sum_j x_{ij}}_{\text{custo de lançamento}} \underbrace{- M \sum_j \max\Big(0,\ n_j^0 + \sum_i x_{ij} - \bar{n}\Big)}_{\text{penalidade de capacidade}}. \tag{10}$$

O primeiro termo é (4) reescrito como ganho: para cada pista, o valor ainda em risco ($V_j S_j$) vezes a chance de que pelo menos um dos novos interceptadores acerte. O segundo termo cobra $\ell_i = 0{,}05$ por interceptador lançado da base (zero para os que já estão em voo). Sem esse custo, como cada lançamento sempre aumenta um pouco o primeiro termo, o ótimo seria lançar todos os interceptadores contra qualquer pista, sem guardar nenhum para as ondas seguintes. O valor 0,05 corresponde a 5 % do valor de uma pista comum: só se lança se a chance de ganho compensar. O terceiro termo impõe a restrição de capacidade (1) como penalidade ($M = 10$), forma conveniente para os métodos de busca, que podem então explorar soluções inviáveis sem quebrar.

3 Revisão da literatura

3.1 Como a revisão foi feita

Esta é uma revisão narrativa estruturada: em vez de enumerar exaustivamente todos os trabalhos publicados, ela organiza a literatura em famílias de técnicas, explica a ideia de cada família e seleciona os trabalhos que a definiram ou que mais se aproximam do nosso problema. A busca usou Google Scholar, IEEE Xplore, ScienceDirect e arXiv, com combinações dos termos *weapon–target assignment*, *dynamic weapon–target assignment*, *threat evaluation and weapon assignment*, *counter-UAV*, *counter-swarm*, *interceptor allocation*, *task allocation*, *auction*, *consensus*, *behavior tree* e *reinforcement learning*, complementada pelas referências dos trabalhos encontrados e por trabalhos brasileiros recentes do Instituto Tecnológico de Aeronáutica. Os critérios de seleção foram: (i) propor ou consolidar uma técnica de alocação usada na prática ou em pesquisa; (ii) tratar defesa contra múltiplas ameaças aéreas; ou (iii) servir de linha de base comum na área. A Tabela 2 situa os marcos no tempo.

Tabela 2. Linha do tempo dos marcos revisados.

Ano	Marco	Ref.
1955–1957	Método húngaro para o problema de atribuição	[20, 21]
1958–1959	Formulação do WTA e heurística de máximo retorno marginal	[1, 2]
1960	Modelos de detecção de alvos com seção radar flutuante	[22]
1986	Prova de que o WTA é NP-completo	[4]
1988	Algoritmo de leilão; filtro IMM para rastreamento	[23, 24]
1990	WTA dinâmico e *shoot–look–shoot*	[3]
2002–2003	Colônia de formigas e algoritmo genético com sementes gulosas para o WTA	[25, 26]
2004	Taxonomia da alocação de tarefas em robótica multiagente	[27]
2006–2007	Revisões do WTA dinâmico e de sistemas TEWA; solução exata e VLSN para o WTA	[5, 6, 28]
2009	Leilões por consenso (CBAA/CBBA)	[29]
2011	Imitação iterativa (DAgger)	[30]
2017	PPO; DeepSets; Transformer	[31–33]
2018	Contra-enxame contra ataques de saturação	[19]
2019	Revisão do WTA; atenção para problemas combinatórios	[7, 34]
2020	Alocação e trajetória por MILP de horizonte deslizante	[14]
2023	Reforço para alocação em enxames; árvores de comportamento contra kamikazes	[12, 16]
2025–2026	Reforço para priorização de interceptações; árvore com utilidade; MIQP contra ameaças aéreas; atenção em grafos + MAPPO	[13, 15, 17, 18]

3.2 Regras gulosas e sistemas TEWA

Conceito: algoritmo guloso

Um algoritmo guloso constrói a solução por uma sequência de escolhas, cada uma a melhor *no momento*, sem nunca voltar atrás. É rápido e simples de entender, mas *míope*: uma escolha boa agora pode impedir escolhas melhores depois. Na alocação, se o primeiro interceptador a decidir fica com uma ameaça que seria muito mais fácil para outro, o segundo fica com o que sobrou (Seção 4.3).

Os primeiros sistemas automatizados de defesa aérea, e muitos dos atuais, são organizados como sistemas de **avaliação de ameaças e atribuição de armas** (TEWA). A decisão é dividida em duas etapas. Na *avaliação de ameaças*, cada alvo detectado recebe uma nota de perigo, calculada a partir de grandezas como o tempo até atingir um ativo defendido, a distância do ponto de maior aproximação a esse ativo e a capacidade inferida do alvo pela sua velocidade, altitude e perfil de voo. Na *atribuição*, os efetores são designados aos alvos na ordem dessas notas, tipicamente o efetor que alcança o alvo mais cedo [6].

Roux e van Vuuren [6] revisaram os modelos de avaliação de ameaças e os algoritmos de atribuição usados em sistemas TEWA e distinguiram abordagens baseadas em regras, em lógica nebulosa, em redes bayesianas e em otimização. Johansson [35] estudou como avaliar o desempenho desses sistemas e mostrou que a qualidade da avaliação de ameaças condiciona fortemente a da atribuição. Uma conclusão recorrente é que, na prática operacional, predominam regras simples e transparentes, porque são rápidas e podem ser auditadas e corrigidas por um operador humano.

Pontos fortes e limitações. As regras gulosas são instantâneas e previsíveis. Mas decidem uma ameaça ou um efetor de cada vez, de modo que o resultado depende da ordem; não percebem

que dois efetores poderiam trocar de alvo com ganho para ambos; e preveem o alvo com velocidade constante.

3.3 Atribuição ótima

Conceito: problema de atribuição linear

Dados n agentes, m tarefas e o custo de cada par, o problema de atribuição linear escolhe os pares de modo que cada agente e cada tarefa apareçam no máximo uma vez e a soma dos custos seja mínima. Embora o número de atribuições possíveis cresça fatorialmente (com 20 agentes e 20 tarefas há $20! \approx 2{,}4 \times 10^{18}$), o problema tem uma estrutura especial que permite resolvê-lo exatamente em *tempo polinomial*: o número de operações cresce como uma potência do tamanho (n^3, no caso do húngaro), e não exponencialmente.

Quando cada alvo recebe no máximo um efetor e o custo de um par não depende dos outros pares, a alocação de uma rodada é um problema de atribuição linear. O **método húngaro**, publicado por Kuhn em 1955 [20] e tornado algoritmo eficiente por Munkres em 1957 [21], foi um dos primeiros algoritmos de otimização combinatória de tempo polinomial. Ele se baseia em duas observações: subtrair uma constante de todos os custos de uma linha ou coluna não muda qual atribuição é a melhor; e, se existe uma atribuição completa usando só custos zero, ela é ótima. O algoritmo subtrai constantes até encontrar essa atribuição. Por ser exato, rápido e amplamente disponível, o húngaro tornou-se a linha de base natural de trabalhos que propõem métodos aprendidos [16].

O **algoritmo de leilão**, proposto por Bertsekas em 1988 [23], resolve o mesmo problema por uma analogia econômica: os agentes dão lances pelas tarefas, cujos preços sobem até que nenhum agente queira trocar. A importância do leilão está em que cada lance usa só os preços correntes, o que permite paralelizar o algoritmo e distribuí-lo entre agentes que só trocam mensagens sobre preços; por isso, ele é a ponte entre a atribuição ótima e a alocação distribuída (Seção 3.6).

Pontos fortes e limitações. Solução exata para a rodada, em milissegundos para as escalas deste trabalho. A limitação é a miopia *temporal*: a solução é ótima para a rodada atual, não para o episódio inteiro; além disso, o custo só pode depender do par, o que exclui efeitos como salvas.

3.4 WTA e retorno marginal

Conceito: problema NP-completo

Um problema é NP-completo quando pertence a uma classe de problemas para os quais não se conhece algoritmo exato de tempo polinomial, e encontrar um seria um avanço de grande impacto na teoria da computação. Na prática, o tempo das melhores soluções exatas conhecidas cresce exponencialmente com o tamanho, de modo que, a partir de algumas dezenas de elementos, recorre-se a heurísticas ou a métodos exatos que funcionam bem só em parte das instâncias.

Quando há probabilidades de abate e um alvo pode receber vários efetores, o objetivo (4) passa a ter produtos e deixa de ser linear. Lloyd e Witsenhausen [4] provaram que esse WTA é NP-completo. O problema foi formulado por Manne [1] e, no ano seguinte, den Broeder et al. [2] propuseram o **máximo retorno marginal** (MMR): a cada passo, designar o par arma–alvo que mais reduz o valor sobrevivente, levando em conta o que já foi designado. O MMR é ótimo

quando todas as armas são idênticas e, até hoje, é o ponto de partida da maioria das heurísticas para o WTA.

Conceito: *branch-and-bound* e busca em vizinhança

Branch-and-bound (ramificação e limitação) resolve problemas combinatórios exatamente por enumeração inteligente: divide o problema em subproblemas (ramos) e descarta os ramos que, por um limitante calculado rapidamente, não podem conter uma solução melhor que a melhor já conhecida. Uma *busca em vizinhança* parte de uma solução e procura uma melhor entre as suas "vizinhas", obtidas por pequenas modificações; uma *busca em vizinhança de grande escala* usa vizinhanças enormes, exploradas por algoritmos eficientes em vez de enumeradas.

Ahuja et al. [28] apresentaram o estudo mais completo de algoritmos exatos e heurísticos para o WTA estático. Para a solução exata, desenvolveram limitantes por programação linear, programação inteira e fluxo em rede e os usaram num *branch-and-bound*, resolvendo à otimalidade instâncias de até 80 armas e 80 alvos. Para instâncias maiores, propuseram uma construção por fluxo em rede seguida de uma busca em vizinhança de grande escala (VLSN), cujos movimentos reatribuem várias armas em cadeia, e obtiveram soluções quase ótimas para instâncias de 200 armas e 200 alvos em segundos. Kline et al. [7] revisaram essas e outras formulações, exatas e heurísticas, e observaram que a maior parte da literatura trata o caso estático, com alvos e probabilidades conhecidos.

Pontos fortes e limitações. O WTA modela explicitamente o valor dos alvos e a chance de sucesso, e por isso entende salvas e a decisão de não engajar. Em contrapartida, precisa de probabilidades de abate que, no nosso problema, não são dadas e precisam ser estimadas (E4), e supõe alvos conhecidos.

3.5 Metaheurísticas

Conceito: metaheurística

Uma metaheurística é um procedimento genérico de busca que não depende da estrutura do problema: basta saber avaliar a qualidade de uma solução candidata. Todas equilibram *diversificação* (visitar regiões novas do espaço de soluções, para não ficar preso num ótimo local) e *intensificação* (refinar as melhores soluções encontradas). Em troca da generalidade, não oferecem garantia de qualidade nem de tempo.

As metaheurísticas formam a família mais numerosa na literatura de WTA [7]. Quatro se destacam, cada uma inspirada num processo natural:

- **Algoritmos genéticos** imitam a seleção natural: mantêm uma população de soluções, selecionam as melhores como pais, combinam-nas por cruzamento e introduzem mutações. Lee et al. [26] mostraram que, no WTA, semear a população inicial com soluções gulosas (o que chamaram de "eugenia gulosa") acelera muito a convergência.
- **Colônias de formigas** imitam o reforço de trilhas de feromônio: soluções são construídas passo a passo, com escolhas guiadas por uma memória coletiva que favorece as decisões presentes nas boas soluções anteriores. Lee et al. [25] combinaram colônia de formigas com mecanismos inspirados no sistema imunológico para o WTA.

- **Recozimento simulado** imita o resfriamento lento de um metal: aceita soluções piores com uma probabilidade que diminui ao longo da busca, o que permite escapar de ótimos locais no início e convergir no fim.
- **Enxame de partículas** imita o movimento de bandos: soluções candidatas se deslocam no espaço de busca, atraídas pela melhor posição de cada uma e pela melhor do bando.

Pontos fortes e limitações. Tratam objetivos e restrições arbitrários — por exemplo, múltiplos tipos de armas, janelas de tempo ou custos não lineares — sem reformular o problema. As limitações são a ausência de garantias e um tempo de execução difícil de limitar, o que pesa num problema que exige uma nova otimização a cada rodada, em fração de segundo.

3.6 Alocação distribuída

Conceito: consenso

Em sistemas distribuídos, consenso é o processo pelo qual agentes que só se comunicam com os vizinhos chegam a uma informação comum. Por exemplo, se cada agente repassa aos vizinhos o maior número que conhece, depois de tantas rodadas quanto o maior número de saltos entre dois agentes, todos conhecem o maior número do grupo.

Em enxames, pode não haver um decisor central, ou a comunicação com ele pode ser lenta ou falhar. Na **alocação distribuída**, cada agente decide a sua tarefa trocando mensagens só com os vizinhos ao alcance. Gerkey e Matarić [27] propuseram a taxonomia mais usada para esses problemas em robótica, que os classifica pelo número de tarefas que um robô executa ao mesmo tempo, pelo número de robôs que uma tarefa exige e pelo horizonte de planejamento; interceptadores descartáveis são o caso mais simples, "uma tarefa por robô, um robô por tarefa, atribuição instantânea".

Choi et al. [29] propuseram o CBAA (*consensus-based auction algorithm*) e a sua extensão para pacotes de tarefas, o CBBA. Cada agente dá lances pelas tarefas com base apenas na sua própria informação; em seguida, numa fase de consenso, os agentes trocam com os vizinhos o que sabem sobre os maiores lances, até que todos concordem sobre quem venceu cada tarefa. Os autores provam que o processo converge para uma atribuição sem conflitos com pelo menos 50 % do valor ótimo, mesmo quando os agentes têm visões inconsistentes da situação — uma propriedade importante quando cada agente vê os alvos por sensores diferentes. Com comunicação completa e uma tarefa por agente, o CBAA produz a mesma atribuição que o guloso global, que fixa repetidamente o par de maior lance.

Pontos fortes e limitações. Robustez a falhas de comunicação e à ausência de decisor central, com garantia de qualidade. A contrapartida é que a qualidade é a de um guloso, não a de um ótimo, e que, com comunicação muito limitada, podem surgir conflitos temporários.

3.7 Alocação dinâmica

Conceito: horizonte deslizante

Numa política de horizonte deslizante, a cada instante resolve-se um problema de otimização que olha para um trecho do futuro, aplica-se só a primeira decisão e, no instante seguinte, resolve-se tudo de novo com a informação atualizada. É a forma prática de lidar com problemas cujo futuro é incerto ou longo demais para ser otimizado de uma vez.

O WTA estático decide tudo de uma vez. Na prática, porém, as armas são disparadas em estágios, e o defensor observa o resultado de cada estágio antes do próximo. Hosein e Athans [3] formalizaram esse **WTA dinâmico** no contexto da defesa contra mísseis balísticos e obtiveram resultados analíticos para casos particulares, e Cai et al. [5] revisaram os modelos e algoritmos dessa linha. Duas lições dessa literatura são diretamente aplicáveis aqui. A primeira é o valor da *reserva*: quando as ameaças chegam em ondas, gastar todos os efetores na primeira onda é subótimo. A segunda é o esquema ***shoot–look–shoot*** ("atirar, olhar, atirar"): em vez de mandar vários efetores contra o mesmo alvo de uma vez (uma *salva*), manda-se um, observa-se o resultado e só se manda outro se o primeiro falhar, o que economiza efetores ao custo de tempo. Como o WTA dinâmico exato é ainda mais difícil que o estático, a aproximação usual é o horizonte deslizante.

Pontos fortes e limitações. Captura a dimensão temporal que o WTA estático ignora. A limitação é que a política ótima depende do que virá depois — quantas ameaças, quando — e isso é desconhecido.

3.8 Alocação integrada à trajetória

Conceito: programação inteira mista e controle preditivo

Um *programa linear inteiro misto* (MILP) é um problema de otimização com função objetivo e restrições lineares em que algumas variáveis só podem assumir valores inteiros — por exemplo, variáveis binárias que valem 1 se o agente i vai contra o alvo j e 0 caso contrário. Se o objetivo é quadrático, fala-se em MIQP. O *controle preditivo* (MPC) aplica o horizonte deslizante ao controle: a cada passo, otimiza a trajetória de um sistema ao longo de um horizonte futuro, a partir de um modelo que prevê como o sistema evolui, aplica só o primeiro comando e repete.

Uma alternativa a separar alocação e guiagem é resolvê-las juntas por otimização matemática: as designações são variáveis binárias, as trajetórias são variáveis contínuas sujeitas a limites de velocidade, aceleração e obstáculos, e o problema é resolvido em horizonte deslizante. Afonso et al. [14] formularam desse modo a alocação de tarefas e o planejamento de trajetória de múltiplos agentes, como um MILP que inclui obstáculos e restrições de conectividade de comunicação entre os agentes. Cruz [15] levou a ideia à neutralização de ameaças aéreas, com um MIQP tridimensional que inclui limites de velocidade e aceleração, obstáculos, o nível de bateria de cada agente na métrica de alocação e uma restrição de alinhamento entre a velocidade final do agente e a do alvo; a proposta foi validada em simulação e em realidade mista, com três drones reais e alvos simulados.

Pontos fortes e limitações. A alocação e as trajetórias resultantes são ótimas e viáveis para o modelo, e restrições operacionais complexas entram de forma natural. As limitações são duas. O número de variáveis binárias cresce com agentes × alvos × passos do horizonte, de modo que o tempo de solução cresce rapidamente com a escala — o próprio autor aponta esse tempo como o obstáculo à operação em malha fechada e sugere aproximações por aprendizado. E a previsão do alvo ao longo do horizonte supõe velocidade constante, justamente a hipótese que uma manobra evasiva viola.

3.9 Arquiteturas comportamentais

Conceito: árvore de comportamento

Uma árvore de comportamento organiza a lógica de decisão de um agente como uma árvore percorrida da raiz às folhas a cada ciclo. As folhas são *condições* ("há uma ameaça detectada?") ou *ações* ("avançar contra a ameaça"). Os nós internos combinam os filhos: um *seletor* tenta os filhos em ordem até que um tenha sucesso (uma lista de alternativas por prioridade); uma *sequência* executa os filhos em ordem enquanto todos têm sucesso (uma lista de passos). Originárias dos jogos digitais, as árvores de comportamento se difundiram na robótica pela legibilidade — é fácil ver por que o agente tomou uma decisão — e pela modularidade — é fácil trocar um ramo sem mexer no resto.

Ricardo Jr et al. [12] aplicaram árvores de comportamento ao engajamento cooperativo de ameaças com enxames de drones: um grupo de defensores protege um drone líder contra ameaças kamikaze, numa arquitetura líder–seguidor, com alocação por distância (cada defensor engaja a ameaça mais próxima) e um simulador físico com dinâmica de multirrotores e controle de baixo nível. Feitosa et al. [13] estenderam essa linha para uma arquitetura sem líder e com decisão baseada em *utilidade*: cada defensor decide avançar sobre uma ameaça quando uma função afim — uma soma ponderada mais uma constante — do tempo de deslocamento, da ameaça do alvo e da ameaça total sobre a defesa é positiva. Os quatro coeficientes da utilidade foram ajustados por algoritmo genético para maximizar o tempo de sobrevivência do enxame. Com 3 defensores armados contra 6 ameaças que voam em linha reta para o centro, e com consciência situacional completa fornecida por um radar central, a defesa resistiu a nove cenários de dificuldade crescente. Num contexto relacionado, Strickland et al. [19] discutiram táticas de contra-enxame contra ataques de saturação, de estratégias de perseguição–evasão a modelos de atrito de Lanchester.

Pontos fortes e limitações. Políticas legíveis, modulares e baratas, que decidem não só contra quem, mas *se* engajar. As limitações são que cada agente decide sozinho, sem otimizar o conjunto, e que os parâmetros ajustados para um tipo de cenário podem não valer para outros.

3.10 Aprendizado de máquina

Conceito: imitação, reforço e invariância à ordem

Na *aprendizagem por imitação*, uma rede neural é treinada para reproduzir as decisões de um especialista a partir de exemplos. No *aprendizado por reforço*, a rede aprende por tentativa e erro, recebendo recompensas e punições pelas consequências das suas decisões, sem especialista. Uma rede é *invariante à ordem* quando a sua saída não muda se os elementos da entrada forem listados em outra ordem — propriedade necessária quando a entrada é um conjunto de ameaças de tamanho

variável.

Os métodos aprendidos substituem a regra escrita à mão por uma rede neural treinada. Há três vertentes.

Imitação. A rede aprende a reproduzir um especialista, tipicamente um algoritmo ótimo, porém mais lento. O problema da imitação ingênua é a *mudança de distribuição*: pequenos erros levam a rede a situações que o especialista nunca produziu, onde ela erra mais, e os erros se acumulam. Ross et al. [30] propuseram o DAgger, que alterna deixar a rede conduzir e pedir ao especialista que rotule as situações que ela visitou, e mostraram que isso limita o acúmulo de erros.

Reforço com política centralizada. Uma rede escolhe a ação a partir do conjunto de ameaças. Como esse número varia, a rede precisa ser invariante à ordem. Isso se obtém por *agregação* — codificar cada ameaça com a mesma rede e combinar os códigos por uma operação simétrica, como a média, ideia formalizada pelo DeepSets [32] — ou por *atenção* [33], um mecanismo em que cada elemento pondera os demais segundo a relevância; Kool et al. [34] mostraram que redes de atenção treinadas por reforço resolvem bem problemas combinatórios como o do caixeiro viajante. O algoritmo de reforço mais usado nessa linha é o PPO [31], que limita o tamanho de cada atualização da política para estabilizar o treino. Liu et al. [16] treinaram uma política de reforço para a alocação de tarefas em enxames e relataram vantagens sobre o método húngaro em tolerância a falhas e tempo de execução. Palmas [17] formulou a priorização de interceptações em defesa contra enxames como um processo de decisão parcialmente observável, treinou uma política de reforço num simulador com efetores de taxa de tiro limitada e relatou menos dano que uma regra manual.

Reforço multiagente. Cada agente tem a sua política e aprende considerando os demais. Jia et al. [18] combinaram redes de atenção em grafos, para a alocação, com o algoritmo multiagente MAPPO, para a manobra, operando em escalas de tempo separadas, e relataram ganho sobre as linhas de base de reforço consideradas.

Pontos fortes e limitações. Decisões rápidas depois de treinadas e capacidade, em princípio, de aprender regularidades que as regras ignoram. As limitações são o custo e a instabilidade do treino, a falta de garantias e a dificuldade de saber se a política generaliza para situações diferentes das de treino. Em todos os trabalhos citados, a comparação é feita com uma ou duas linhas de base, em simulador próprio.

3.11 Guiagem e rastreamento

A eficácia de uma designação depende de dois componentes fora da alocação. A **guiagem terminal** determina se o interceptador consegue fechar o contato com o alvo; as leis clássicas e suas variantes são tratadas em [36–38], inclusive para drones [39]. O **rastreamento** transforma medidas de radar em pistas; os padrões são o filtro IMM [24], a associação de dados entre medidas

Tabela 3. Síntese das famílias de técnicas de alocação.

Família	Ideia	Pontos fortes	Limitações
Gulosa / TEWA	melhor escolha local, uma de cada vez	instantânea, auditável	míope, depende da ordem
Atribuição ótima	ótimo exato de um par por alvo	exata e rápida	míope no tempo; não entende salvas
WTA	maximizar valor esperado destruído	modela valor, salvas e a decisão de esperar	precisa de probabilidades de abate
Metaheurística	busca genérica guiada pela qualidade	objetivos arbitrários	sem garantia; tempo imprevisível
Distribuída	lances locais + consenso	sem decisor central; robusta a falhas	qualidade de guloso
Dinâmica	reserva, observar antes de reengajar	considera o tempo	depende do futuro desconhecido
Integrada à trajetória	otimizar alocação e trajetória juntas	viável e ótima para o modelo	custo cresce com a escala; previsão de velocidade constante
Comportamental	árvore de regras com utilidade	legível, decide se engaja	sem otimização conjunta; parâmetros específicos
Aprendizado	rede treinada por imitação ou reforço	decisão rápida; pode aprender regularidades	treino caro; sem garantia; generalização incerta

e pistas [40] e os modelos de detecção de alvos cuja seção radar flutua [22]. Neste trabalho, ambos são componentes fixos do simulador, iguais para todos os métodos (Seção 5).

3.12 Síntese

A Tabela 3 resume as famílias, e a Tabela 4 compara os trabalhos mais próximos do nosso. Três observações motivam o restante do artigo. Primeiro, todas as famílias, de um modo ou de outro, preveem o alvo com velocidade constante ou supõem que ele é conhecido; nenhuma foi avaliada contra alvos evasivos vistos por radar. Segundo, as famílias otimizam critérios diferentes — tempo, valor esperado, utilidade, recompensa —, e não é óbvio qual critério produz menos dano no problema real. Terceiro, os trabalhos que propõem métodos novos comparam-nos com uma ou duas linhas de base, em escalas pequenas e em simuladores diferentes, de modo que não é possível ordenar as famílias a partir da literatura.

4 Métodos implementados

Esta seção descreve, um a um, os 29 métodos implementados. Começamos por um catálogo e por um exemplo numérico que mostra por que famílias diferentes chegam a decisões diferentes; em seguida, detalhamos cada família.

4.1 Interface comum

Para que a comparação seja justa, todos os métodos usam a mesma interface. A cada rodada de decisão, recebem o mesmo quadro de pistas e as mesmas zonas prioritárias, e devolvem uma ação — uma pista ou a ação nula — para cada interceptador da fila. Há dois estilos de decisão:

- Os métodos **individuais** (gulosos e árvores de comportamento) decidem um interceptador por vez, na ordem da fila, sem considerar os demais da rodada. A pista escolhida fica indisponível para os seguintes.
- Os métodos **conjuntos** resolvem a rodada inteira — todos os interceptadores da fila contra

Tabela 4. Comparação dos trabalhos mais próximos. Escala: defensores × ameaças. Informação: o que o decisor observa. Alvo: modelo de movimento das ameaças.

Trabalho	Método	Escala	Informação	Alvo	Comparação
Ricardo Jr et al. [12]	árvore, alocação por distância	poucos	completa	kamikaze em linha reta	simulador físico próprio
Feitosa et al. [13]	árvore com utilidade, GA	3×6	completa (radar central)	linha reta	9 cenários, sem linha de base
Afonso et al. [14]	MILP de horizonte deslizante	poucos	completa	tarefas fixas	cenários de demonstração
Cruz [15]	MIQP de horizonte deslizante	3×3	posição e velocidade por radar	velocidade constante	simulação e realidade mista
Liu et al. [16]	reforço profundo	enxame	completa	–	húngaro
Palmas [17]	reforço profundo	configurável	ruidosa e parcial	rotas pré-definidas	regra manual
Jia et al. [18]	atenção em grafos + MAPPO	enxame	–	intrusão dinâmica	linhas de base de reforço
Este trabalho	29 métodos de 8 famílias	até 60×48	radar com falsos alarmes, relevo e vieses	4 classes reais, com iscas, contra ativos reais	todos contra todos, testes pareados

todas as pistas — e entregam a cada interceptador a sua parte da solução.

Nenhum método acessa a posição verdadeira das ameaças nem sabe quais pistas são falsas ou quais são iscas. Exceto M1 e M8, todos aplicam o filtro de plausibilidade (E1).

4.2 Catálogo

A Tabela 5 lista os 29 métodos, numerados de M1 a M29, com a origem, o funcionamento e a adaptação de cada um em uma linha. As subseções seguintes detalham cada um.

4.3 Exemplo ilustrativo

Antes do detalhe, um exemplo pequeno mostra por que as famílias chegam a decisões diferentes. Há três interceptadores (A, B e C) e três pistas; a pista 3 está numa zona prioritária, de modo que os pesos são $w = (1, 1, 10)$. Os tempos de interceptação (em segundos) e as probabilidades de sucesso dadas por (9) são

$$c = \begin{pmatrix} 10 & 12 & 50 \\ 11 & 60 & 70 \\ 40 & 15 & 30 \end{pmatrix}, \qquad q = \begin{pmatrix} 0{,}836 & 0{,}833 & 0{,}782 \\ 0{,}835 & 0{,}769 & 0{,}756 \\ 0{,}795 & 0{,}829 & 0{,}809 \end{pmatrix},$$

com uma linha por interceptador e uma coluna por pista. A Tabela 6 traz a solução de cada família, obtida com as próprias implementações.

O guloso por proximidade, decidindo A primeiro, dá a A a pista 1 (10 s), embora A seja quase tão rápido para a pista 2 (12 s); com isso, B — que só é rápido para a pista 1 — fica com a pista 2, a 60 s. O húngaro enxerga a rodada inteira, troca A e B de pista e reduz o tempo total de 100 para 53 s. O CBAA e o MMR também fixam primeiro o par mais atraente e chegam à mesma solução que o guloso, embora por motivos diferentes: o MMR começa pela pista prioritária, cujo ganho ($10 \times 0{,}809$) domina os demais. O WTA exato encontra o maior valor esperado destruído

Tabela 5. Catálogo dos métodos implementados. *Ind.*: decide um interceptador por vez; *conj.*: resolve a rodada inteira. As referências são as de origem; a última coluna resume o que mudou para o nosso problema.

#	Método	Origem	Como funciona	Adaptação
Regras simples				
M1	Aleatório	–	pista sorteada, sem filtro, nunca troca (ind.)	piso de desempenho
M2	Guloso (próximo)	[7]	menor c_{ij} (ind.)	–
M3	Guloso (urgente)	[7]	menor folga até o solo (ind.)	folga estimada só pelo radar
M4	Guloso (prioridade)	[7]	menor c_{ij}/w_j (ind.)	peso inferido pela geometria (E3)
M5	TEWA sequencial	[6]	ordena por ameaça, designa o mais rápido (conj.)	ameaça = peso / tempo até a zona
Atribuição ótima (um interceptador por pista)				
M6	Húngaro	[20, 21]	atribuição de mínimo tempo total (conj.)	custo com histerese (E2)
M7	Húngaro ponderado	[20]	idem, tempo dividido pelo peso (conj.)	especialista das redes
M8	Húngaro pond. sem filtro	[20]	M7 sem o filtro (E1)	mede o valor do filtro
M9	Leilão	[23]	lances e preços até o equilíbrio (conj.)	versão assimétrica sem escalonamento de ε
Alocação distribuída				
M10	Guloso global	[29]	fixa o par de maior lance, repete (conj.)	forma centralizada do CBAA
M11	CBAA	[29]	lances locais + consenso de máximo (conj.)	comunicação completa via estação central
M12	CBAA com comunic. local	[29]	idem, grafo de 3 km, 3 rodadas	conflitos resolvidos por ordem de chegada
WTA (objetivo de valor esperado (10))				
M13	MMR	[2]	insere o par de maior ganho marginal (conj.)	custo de lançamento decide esperar
M14	Húngaro sobre WTA	[28]	solução exata de (10) com $\bar{n} = 1$ (conj.)	coluna “esperar” por interceptador
M15	MILP (HiGHS)	[14, 15]	mesmo problema como programa inteiro	parte de alocação do MPC inteiro
M16	Busca local (VLSN)	[28]	melhora o MMR por realocação e troca	vizinhança polinomial
Metaheurísticas (objetivo (10), semeadas com o MMR)				
M17	Algoritmo genético	[26]	torneio, cruzamento uniforme, mutação, elitismo	semente gulosa (“eugenia”)
M18	Colônia de formigas	[25]	construção probabilística guiada por feromônio	opção “esperar” como caminho
M19	Recozimento simulado	[7]	aceita pioras com probabilidade decrescente	parte do MMR
M20	Enxame de partículas	[7]	partículas contínuas decodificadas por chave	codificação por chaves aleatórias
Alocação dinâmica				
M21	Húngaro com reserva	[3]	M7, mas guarda 20 % dos interceptadores	reserva só usada por pistas prioritárias
Arquiteturas comportamentais				
M22	Árvore (distância)	[12]	engaja a mais próxima a menos de 4 km (ind.)	sem líder–seguidor
M23	Árvore (utilidade)	[13]	avança se a utilidade afim > 0 (ind.)	sem ramo de reforço; GA no treino
Políticas aprendidas (imitam M7 e/ou otimizam o dano)				
M24	Imitação, DeepSets	[30, 32]	DAgger sobre M7	amostragem estratificada das esperas
M25	Imitação, atenção	[30, 34]	idem, codificador Transformer	idem
M26	Imitação+PPO, DeepSets	[31]	M24 refinada por reforço	perda auxiliar de legitimidade
M27	Imitação+PPO, atenção	[18, 31]	M25 refinada por reforço	idem
M28	PPO, DeepSets	[16, 17]	reforço do zero	política compartilhada
M29	PPO, atenção	[18]	reforço do zero	política compartilhada

Tabela 6. Exemplo ilustrativo: solução de cada família para a mesma instância (pista 3 prioritária). F é o valor esperado destruído (10) sem custo de lançamento.

Método	Designação	Tempo total (s)	F	Por quê
Guloso próximo (ordem A,B,C)	A→1, B→2, C→3	100	9,691	A decide primeiro
Guloso prioridade	A→3, B→1, C→2	76	9,484	o mais rápido vai à prioritária
Guloso global / CBAA	A→1, B→2, C→3	100	9,691	fixa o melhor par primeiro
MMR	A→1, B→2, C→3	100	9,691	começa pela prioritária
Húngaro / leilão	A→2, B→1, C→3	53	9,753	ótimo do tempo total
WTA exato (M14, M15)	A→2, B→1, C→3	53	9,753	ótimo do valor esperado

(9,753 contra 9,691), o que confirmamos enumerando as seis atribuições possíveis. O guloso por prioridade, ao dividir os tempos pelos pesos, manda o interceptador mais rápido de todos (A) à pista prioritária e piora o restante.

4.4 Regras simples (M1–M5)

M1 – Aleatório

Ideia. Medir o que se obtém sem usar informação alguma do quadro de radar.

Como funciona. Cada interceptador ocioso é lançado contra uma pista sorteada uniformemente entre as livres, sem filtro de plausibilidade, e nunca troca de alvo.

O que esperar. É o *piso* de desempenho: um método que não o supera não está extraindo informação útil das pistas.

M2–M4 – Gulosos

Ideia. Cada interceptador escolhe, sozinho, a pista que lhe parece melhor por um critério simples.

Origem. Regras de despacho comuns em sistemas reais [6, 7].

Como funciona. O interceptador da vez escolhe, entre as pistas livres e plausíveis,

$$\text{M2: } \arg\min_j c_{ij}, \qquad \text{M3: } \arg\min_j (t_j^G - c_{ij}), \qquad \text{M4: } \arg\min_j \frac{c_{ij}}{w_j}, \tag{11}$$

em que $\arg\min_j$ indica a pista j que minimiza a expressão. M2 (*próximo*) escolhe a pista que alcança mais cedo. M3 (*urgente*) escolhe a pista com a menor folga entre o tempo que ela leva até o solo, $t_j^G = h_j^{\mathrm{AGL}}/(-v_j^z)$ (altura dividida pela velocidade de descida, infinito se a pista não desce), e o tempo de interceptação: uma folga pequena significa que é preciso agir já. M4 (*prioridade*) divide o tempo pelo peso da zona, de modo que pistas prioritárias parecem dez vezes mais próximas. A pista escolhida fica indisponível para os interceptadores seguintes da fila.

Propriedades. Custo de $O(K)$ operações por interceptador, para K pistas. Nenhuma garantia de qualidade; o resultado depende da ordem em que os interceptadores decidem (Tabela 6).

Adaptações. O tempo até o solo (M3) e o peso (M4) são inferidos só pelo radar, porque o

defensor não conhece o ponto de impacto.

O que esperar. Rápidos e razoáveis quando há folga de interceptadores. M3 é o mais frágil diante de manobras, porque a velocidade de descida estimada é ruidosa em zigue-zague e em voo baixo.

M5 – TEWA sequencial

Ideia. Primeiro decidir *quais ameaças importam mais*; depois designar a cada uma o melhor interceptador disponível.

Origem. Estrutura clássica dos sistemas de avaliação de ameaças e atribuição de armas [6].

Como funciona. Cada pista recebe uma nota de ameaça, que cresce quanto mais perto ela estiver de uma zona prioritária:

$$\theta_j = \begin{cases} w_P/(1+t_j^Z), & t_j^Z < \infty, \\ w_O/10^3, & \text{caso contrário.} \end{cases} \tag{12}$$

As pistas são visitadas em ordem decrescente de θ_j, e cada uma recebe, entre os interceptadores ainda livres, o de menor custo c_{ij}. O processo para quando acabam os interceptadores da rodada.

Propriedades. Custo de $O(K \log K + nK)$ (ordenar as pistas e procurar o melhor interceptador para cada uma). Nenhuma garantia.

Adaptações. A ameaça usa o tempo até a zona prioritária (E3) no lugar do tempo até o ativo defendido.

O que esperar. Protege bem as zonas prioritárias, mas pode gastar interceptadores distantes com ameaças de nota alta enquanto deixa passar ameaças de nota baixa e fáceis de pegar; herda os erros de t_j^Z quando a rota passa perto de ativos que não são o alvo.

4.5 Atribuição ótima (M6–M9)

M6–M8 – Método húngaro

Ideia. Escolher, para a rodada inteira, o conjunto de pares interceptador–pista de menor custo total, considerando todos ao mesmo tempo.

Origem. Kuhn [20], que o batizou em homenagem a trabalhos de matemáticos húngaros, e Munkres [21].

Como funciona. O método resolve o problema de atribuição

$$\min_x \sum_{i,j} \frac{c_{ij}}{w_j^\beta} x_{ij} \quad \text{s.a.} \quad \sum_j x_{ij} \le 1, \quad \sum_i x_{ij} \le 1, \quad x_{ij} \in \{0,1\}, \tag{13}$$

isto é, minimiza a soma dos custos dos pares escolhidos, com cada interceptador e cada pista usados no máximo uma vez. O problema pode ser *retangular* (mais interceptadores que pistas, ou o contrário). A intuição do algoritmo é a seguinte: subtrair uma constante de todos os

custos de uma linha (ou de uma coluna) não muda qual atribuição é a melhor, porque todas as atribuições usam exatamente uma posição daquela linha. O algoritmo subtrai constantes — chamadas *potenciais* — até que exista uma atribuição completa usando só posições de custo zero; como os custos restantes são não negativos, essa atribuição é ótima. Encontrar a atribuição entre os zeros é feito por *caminhos alternantes*, que trocam pares já escolhidos por outros até acomodar mais um interceptador. Um resultado de teoria de otimização (a *unimodularidade total* da matriz de restrições) garante que, nesse problema, a solução ótima com variáveis contínuas já é inteira, o que explica por que ele é fácil apesar de combinatório.

Propriedades. Exato para (13), em $O(n^2K)$ operações para n interceptadores e K pistas.

Adaptações. M6 usa $\beta = 0$ (minimiza o tempo total); M7 usa $\beta = 1$ (divide o tempo pelo peso, dando preferência às pistas prioritárias); M8 é o M7 sem o filtro (E1), para medir quanto o filtro vale. A pista atual de um interceptador em voo entra com a histerese (E2). M7 é também o especialista imitado pelas redes neurais. Usamos a implementação de `scipy.optimize.linear_sum_assignment`.

O que esperar. A solução é ótima para a rodada, mas não para o episódio: não considera ameaças que ainda vão aparecer, e o custo é uma previsão de velocidade constante.

M9 – Leilão de Bertsekas

Ideia. Resolver o mesmo problema de atribuição como um mercado: os interceptadores "compram" as pistas, e os preços sobem até que ninguém queira trocar.

Origem. Bertsekas [23].

Como funciona. Cada pista j tem um preço p_j, inicialmente zero, e cada par tem um benefício $a_{ij} = -c_{ij}/w_j$ (quanto menor o custo, maior o benefício). O *valor líquido* de uma pista para um interceptador é o benefício menos o preço. Um interceptador sem pista encontra a pista de maior valor líquido, j_i, com valor $v^{(1)}$, e a segunda melhor, com valor $v^{(2)}$, e dá o lance

$$p_{j_i} \leftarrow p_{j_i} + v^{(1)} - v^{(2)} + \varepsilon. \tag{14}$$

Ou seja, eleva o preço da pista até o ponto em que ela ainda lhe vale tanto quanto a segunda opção, mais um pequeno incremento ε. Se a pista tinha dono, o dono anterior fica sem pista e volta a dar lances (Algoritmo 1). O incremento $\varepsilon > 0$ garante que os preços sempre sobem e que o processo termina.

Propriedades. Ao final, cada interceptador está com uma pista cujo valor líquido está a menos de ε do melhor disponível para ele (condição chamada de ε-*folga complementar*), e o custo total fica a menos de $n\varepsilon$ do ótimo [23]. Com ε pequeno, o leilão chega à mesma solução que o húngaro. Como cada lance só usa os preços correntes, o algoritmo se presta a implementação paralela ou distribuída.

Adaptações. Usamos os mesmos benefícios de M7, para que os dois sejam diretamente comparáveis. O caso *assimétrico* (números diferentes de interceptadores e pistas) exige cuidado. A prática usual de começar com ε grande e reduzi-lo (escalonamento de ε) acelera

o leilão, mas, no caso assimétrico, deixa preços altos em pistas que terminam sem dono e perde a garantia de otimalidade — uma primeira versão com escalonamento coincidiu com o húngaro em só 10 % de instâncias aleatórias. A versão final começa com preços nulos e usa um único ε pequeno.

Parâmetros. $\varepsilon = 10^{-3}\,\text{amplitude}(a)/(n+1)$, em que a amplitude é a diferença entre o maior e o menor benefício.

Algoritmo 1 Leilão direto para atribuição assimétrica (M9).

Entrada: benefícios a_{ij} $(n \le m)$, $\varepsilon > 0$

1: $p_j \leftarrow 0$ para todo j; todos os i livres
2: **while** existe i livre com algum a_{ij} finito **do**
3: $\quad j_i \leftarrow \arg\max_j (a_{ij} - p_j)$; $\quad v^{(1)} \leftarrow a_{ij_i} - p_{j_i}$; $\quad v^{(2)} \leftarrow \max_{j \neq j_i}(a_{ij} - p_j)$
4: $\quad p_{j_i} \leftarrow p_{j_i} + v^{(1)} - v^{(2)} + \varepsilon$
5: $\quad$ designa j_i a i; se j_i tinha dono k, k volta a ficar livre

Saída: designação $i \mapsto j_i$

4.6 Alocação distribuída (M10–M12)

M10–M12 – Guloso global e CBAA

Ideia. Permitir que cada interceptador escolha a sua pista trocando mensagens só com os vizinhos, sem decisor central, e ainda assim evitar que dois interceptadores persigam a mesma pista.

Origem. Choi, Brunet e How [29].

Como funciona. Cada interceptador i atribui a cada pista um *lance* $b_{ij} = w_j / \max(c_{ij}, 1)$, maior para pistas prioritárias e próximas, e guarda três informações: a sua tarefa x_i, o maior lance que conhece para cada pista, y_{ij}, e quem deu esse lance, z_{ij}. Duas fases se alternam (Algoritmo 2):

- na fase de *lances*, um interceptador sem tarefa escolhe a pista em que o seu lance supera o maior lance que ele conhece, $x_i = \arg\max_j \{b_{ij} : b_{ij} > y_{ij}\}$, e se declara vencedor dela;
- na fase de *consenso*, cada interceptador compara o que sabe com o que sabem os vizinhos do grafo de comunicação e adota, para cada pista, o maior lance conhecido,

$$y_{ij} \leftarrow \max_{k \in \mathcal{N}_i \cup \{i\}} y_{kj}, \tag{15}$$

com desempate pelo menor índice. Se descobre que alguém deu um lance maior pela sua pista, desiste dela e volta a dar lances na rodada seguinte.

O processo para quando ninguém muda de tarefa.

Propriedades. Como a informação sobre o maior lance se propaga um vizinho por rodada, o CBAA converge em no máximo (número de agentes) × (diâmetro do grafo) rodadas, onde o diâmetro é o maior número de saltos entre dois agentes. O resultado não tem conflitos

e vale pelo menos 50 % do ótimo [29]. Com grafo completo (todos falam com todos), o resultado coincide com o do *guloso global*: fixar o par de maior lance entre todos, remover o interceptador e a pista, e repetir.

Adaptações. Implementamos as duas formas: o guloso global centralizado (M10) e o CBAA iterativo com grafo completo (M11), representando a comunicação via estação central de [13]; a Seção 8.1 confirma que dão a mesma solução. M12 restringe a comunicação a interceptadores a até 3 km uns dos outros e a 3 rodadas de consenso por decisão. Com isso, interceptadores de bases diferentes podem acreditar que ganharam a mesma pista; o simulador resolve o conflito pela ordem de chegada, e o perdedor espera a próxima rodada — que é o custo real de uma visão inconsistente.

Parâmetros. M11: até 100 rodadas de consenso; M12: alcance de 3 km e 3 rodadas.

O que esperar. O guloso global é melhor que os gulosos individuais (não depende da ordem da fila), mas pode perder para o húngaro (Tabela 6). M12 mede quanto se perde com comunicação limitada.

Algoritmo 2 CBAA: consenso sobre lances, uma tarefa por agente (M11, M12).

Entrada: lances b_{ij}, grafo $\mathcal{G}$, máximo de rodadas R

$x_i \leftarrow \varnothing$, $y_{ij} \leftarrow -\infty$, $z_{ij} \leftarrow \varnothing$ para todo i, j
for $r = 1, \ldots, R$ **do**
  for cada i com $x_i = \varnothing$ **do** ▷ lances
    $H \leftarrow \{j : b_{ij} > y_{ij}\}$; se $H \neq \varnothing$: $x_i \leftarrow \arg\max_{j \in H} b_{ij}$, $y_{ix_i} \leftarrow b_{ix_i}$, $z_{ix_i} \leftarrow i$
  for cada i **do** ▷ consenso
    $y_{ij}, z_{ij} \leftarrow$ maior lance e vencedor entre $\mathcal{N}_i \cup \{i\}$, para todo j
    se $z_{ix_i} \neq i$: $x_i \leftarrow \varnothing$
  se nada mudou, pare

Saída: tarefa x_i de cada agente

4.7 WTA: retorno marginal, solução exata e busca local (M13–M16)

Os métodos desta família otimizam o valor esperado destruído (10), e não o tempo. Isso traz duas diferenças práticas em relação aos anteriores: eles podem decidir *não lançar*, quando nenhum par compensa o custo de lançamento; e entendem salvas, isto é, sabem que o segundo interceptador contra a mesma pista vale menos que o primeiro, porque só conta se o primeiro falhar.

M13 – Máximo retorno marginal (MMR)

Ideia. Designar um interceptador de cada vez ao par que mais aumenta o valor esperado destruído, considerando o que já foi designado.

Origem. den Broeder, Ellison e Emerling [2].

Como funciona. O *ganho marginal* de designar i à pista j é quanto o valor esperado destruído aumenta com essa designação, descontado o custo de lançamento:

$$\Delta_{ij} = V_j \, \tilde{S}_j \, q_{ij} - \ell_i, \tag{16}$$

em que $\tilde{S}_j$ é a chance de a pista ainda sobreviver dadas as designações feitas até agora. O algoritmo escolhe o par livre de maior ganho, designa-o e atualiza a sobrevivência da pista, $\tilde{S}_j \leftarrow \tilde{S}_j(1 - q_{ij})$ — de modo que um segundo interceptador na mesma pista terá ganho menor —, e repete até que nenhum ganho seja positivo (Algoritmo 3). Esse critério de parada é o que implementa a decisão de esperar.

Propriedades. Ótimo quando todas as armas são idênticas; em geral, uma heurística de bom desempenho e ponto de partida de métodos melhores [28]. Custo de $O(n^2K)$.

Adaptações. Valor e probabilidade de sucesso dados por (E3) e (E4).

Parâmetros. P_T, τ_q e ℓ_i de (E4).

O que esperar. A qualidade depende do modelo de sucesso (9): se as manobras reduzem a chance real de contato de modo diferente do previsto pelo modelo, a ordenação dos ganhos fica errada.

Algoritmo 3 Máximo retorno marginal (M13).

Entrada: q_{ij}, V_j, S_j, ℓ_i, n_j^0, $\bar{n}$

1: $\tilde{S}_j \leftarrow S_j$, $n_j \leftarrow n_j^0$, todos os i livres
2: **loop**
3: $\quad (i^\star, j^\star) \leftarrow \arg\max\{V_j \tilde{S}_j q_{ij} - \ell_i : \ i \text{ livre}, \ n_j < \bar{n}\}$
4: $\quad$ se o ganho é ≤ 0, pare
5: $\quad x_{i^\star j^\star} \leftarrow 1; \quad \tilde{S}_{j^\star} \leftarrow \tilde{S}_{j^\star}(1 - q_{i^\star j^\star}); \quad n_{j^\star} \leftarrow n_{j^\star} + 1$

Saída: $\boldsymbol{x}$

M14, M15 – WTA exato (húngaro e MILP)

Ideia. Quando cada pista recebe no máximo um interceptador, o objetivo de valor esperado se torna linear e pode ser otimizado exatamente.

Origem. Formulação exata do WTA [28]; M15 representa a família de otimização inteira mista de [14, 15].

Como funciona. Com $\bar{n} = 1$, cada pista tem um único fator no produto de (10), e o valor destruído por designar i a j passa a ser simplesmente $V_j S_j q_{ij}$. O problema fica

$$\max_x \sum_{i,j} (V_j S_j q_{ij} - \ell_i) x_{ij} \quad \text{s.a.} \quad \sum_j x_{ij} \leq 1, \quad \sum_i x_{ij} \leq \bar{n} - n_j^0, \quad x_{ij} \in \{0, 1\}, \tag{17}$$

que é de novo um problema de atribuição, agora maximizando valor em vez de minimizar tempo. Resolvemo-lo de duas formas. M14 usa o húngaro: replica a coluna de cada pista conforme a sua capacidade restante e acrescenta, para cada interceptador, uma coluna "esperar" de valor nulo, de modo que esperar seja escolhido quando nenhum par tem valor positivo. M15 escreve (17) como *programa linear inteiro* — um problema de otimização linear em que as variáveis devem ser inteiras — e o resolve com o solver aberto HiGHS, que usa *ramificação e corte*: resolve relaxações contínuas, divide o problema em ramos quando a solução não é inteira e acrescenta restrições (cortes) que eliminam soluções fracionárias.

Propriedades. Ambos encontram o ótimo de (17); devem atingir o mesmo valor, diferindo só no custo computacional e, quando há empates, em qual das soluções ótimas escolhem.

Adaptações. Em [15], a alocação está acoplada ao planejamento da trajetória, com obstáculos e bateria; aqui reproduzimos só a parte de alocação, com a previsão de velocidade constante embutida em c_{ij}.

Parâmetros. M15: limite de 5 s por rodada.

O que esperar. Diferem do húngaro ponderado (M7) no critério: valor esperado em vez de tempo, e possibilidade de esperar.

M16 – Busca local (VLSN simplificada)

Ideia. Partir de uma boa solução gulosa e melhorá-la por pequenas modificações enquanto houver ganho.

Origem. Busca em vizinhança de grande escala de Ahuja et al. [28].

Como funciona. Uma *vizinhança* é o conjunto de soluções obtidas a partir da atual por uma modificação simples. Usamos duas: *realocação* (um interceptador muda para outra pista viável ou passa a esperar) e *troca* (dois interceptadores trocam de pista entre si). A partir da solução do MMR, avaliamos todas as soluções das duas vizinhanças, adotamos a de maior valor de (10) se ela for melhor que a atual, e repetimos até não haver melhoria.

Propriedades. Termina num *ótimo local*: uma solução que nenhuma realocação ou troca isolada melhora. Nunca é pior que o MMR.

Adaptações. A VLSN original usa vizinhanças exponencialmente grandes (reatribuições em cadeia) exploradas por algoritmos de fluxo em rede; a nossa usa vizinhanças de tamanho polinomial, avaliadas de forma vetorizada.

Parâmetros. Até 30 iterações.

4.8 Metaheurísticas (M17–M20)

As quatro metaheurísticas otimizam o mesmo objetivo (10). Todas representam uma solução por um vetor $\boldsymbol{g} \in \{-1, 0, \ldots, K-1\}^{|\mathcal{D}_t|}$, com uma posição por interceptador da fila contendo o índice da pista designada (-1 significa esperar), e só consideram pares viáveis. Todas incluem a solução do MMR no ponto de partida, de modo que nunca terminam abaixo dela. Por serem estocásticas, usam sementes aleatórias fixadas pelo instante da rodada, para que os resultados sejam reprodutíveis.

M17 – Algoritmo genético

Ideia. Imitar a seleção natural: manter uma *população* de soluções, combinar as melhores para gerar novas e introduzir variações aleatórias.

Origem. Lee, Su e Lee [26], que semeiam a população com soluções gulosas (“eugenia gulosa”).

Como funciona. A cada *geração*: (i) *seleção por torneio* — sorteiam-se três soluções e a melhor delas é escolhida como pai, o que favorece as boas sem descartar as demais; (ii) *cruzamento uniforme* — o filho herda a designação de cada interceptador de um dos dois pais, sorteado com igual chance; (iii) *mutação* — cada designação do filho é trocada por uma opção viável sorteada com pequena probabilidade, o que mantém a diversidade; (iv) *elitismo* — as duas melhores soluções passam intactas para a próxima geração, para não perder o melhor já encontrado. A qualidade (*aptidão*) de cada solução é (10).

Parâmetros. População de 40, 40 gerações, torneio de 3, probabilidade de mutação $1/|\mathcal{D}_t|$, elitismo de 2.

M18 – Colônia de formigas

Ideia. Imitar formigas que, ao encontrar comida, reforçam com feromônio o caminho percorrido; caminhos bons acumulam mais feromônio e atraem mais formigas.

Origem. Lee, Lee e Su [25].

Como funciona. Cada *formiga* constrói uma solução visitando os interceptadores em ordem aleatória e escolhendo, para cada um, uma opção j (uma pista ou esperar) com probabilidade

$$P_{ij} = \frac{\tau_{ij}^{\alpha}\eta_{ij}^{\beta}}{\sum_{k}\tau_{ik}^{\alpha}\eta_{ik}^{\beta}}, \tag{18}$$

que combina duas informações: a *atratividade heurística* $\eta_{ij} = \max(V_j S_j q_{ij} - \ell_i, 10^{-6})$, o ganho do par se ele fosse o único, e o *feromônio* τ_{ij}, que acumula a experiência das iterações anteriores. Os expoentes α e β regulam o peso de cada uma. Pistas já no limite de capacidade recebem probabilidade nula. Ao fim de cada iteração, todo o feromônio *evapora* parcialmente (o que permite esquecer escolhas ruins) e a melhor solução encontrada é reforçada: $\tau \leftarrow (1-\rho)\tau + \rho F^{\star}\mathbf{1}_{\boldsymbol{g}^{\star}}$, em que $\mathbf{1}_{\boldsymbol{g}^{\star}}$ marca os pares da melhor solução e $F^{\star}$ é o seu valor.

Adaptações. A opção "esperar" é tratada como um caminho com atratividade fixa ($\eta_{i,-1} = 0{,}05$).

Parâmetros. 16 formigas, 15 iterações, $\alpha = 1$, $\beta = 2$, taxa de evaporação $\rho = 0{,}1$.

M19 – Recozimento simulado

Ideia. Imitar o resfriamento lento de um metal: no início, o sistema "quente" aceita mudanças que pioram a solução, o que permite escapar de ótimos locais; à medida que esfria, torna-se cada vez mais exigente.

Origem. Metaheurística de uso amplo no WTA [7].

Como funciona. Partindo do MMR, propõe-se a cada passo uma realocação aleatória de um interceptador. Uma melhoria é sempre aceita; uma piora $\Delta F < 0$ é aceita com probabilidade $e^{\Delta F/T}$, em que T é a *temperatura*: com T alta, quase toda piora é aceita; com T baixa, quase nenhuma. A temperatura diminui a cada passo, e guarda-se a melhor solução visitada.

Parâmetros. 1500 movimentos, temperatura inicial $T_0 = 0{,}2|F_0|$ (proporcional ao valor da solução inicial) e resfriamento $T \leftarrow 0{,}995\,T$.

M20 – Enxame de partículas

Ideia. Imitar um bando: cada solução candidata (uma *partícula*) se move pelo espaço de soluções, atraída pela melhor posição que ela própria já visitou e pela melhor posição encontrada pelo bando.

Origem. Metaheurística de uso amplo no WTA [7].

Como funciona. Cada partícula tem uma posição $\boldsymbol{X}$ e uma velocidade $\boldsymbol{V}$, atualizadas por

$$\boldsymbol{V} \leftarrow \omega\boldsymbol{V} + c_1\boldsymbol{r}_1 \odot (\boldsymbol{X}^p - \boldsymbol{X}) + c_2\boldsymbol{r}_2 \odot (\boldsymbol{X}^g - \boldsymbol{X}), \qquad \boldsymbol{X} \leftarrow \mathrm{clip}_{[0,1]}(\boldsymbol{X} + \boldsymbol{V}), \tag{19}$$

em que $\boldsymbol{X}^p$ é a melhor posição da própria partícula, $\boldsymbol{X}^g$ a melhor do bando, $\boldsymbol{r}_1, \boldsymbol{r}_2$ números aleatórios uniformes e $\odot$ o produto elemento a elemento. O primeiro termo é a *inércia* (manter a direção), o segundo a atração *individual* e o terceiro a atração *social*.

Adaptações. O enxame de partículas trabalha com posições contínuas, mas a alocação é discreta. Usamos a codificação por *chaves aleatórias*: cada partícula é uma tabela de números em $[0, 1]$, com uma linha por interceptador e uma coluna por opção (cada pista e esperar); para obter a alocação, cada interceptador recebe a opção viável de maior número na sua linha. Uma partícula começa na codificação da solução do MMR.

Parâmetros. 20 partículas, 25 iterações, inércia $\omega = 0{,}7$, $c_1 = c_2 = 1{,}5$.

O que esperar das metaheurísticas. Elas só podem superar o MMR se o objetivo (10) tiver estrutura que a escolha gulosa perde; em troca, custam uma a duas ordens de grandeza mais tempo por rodada (Seção 8.1).

4.9 Alocação dinâmica (M21 e variantes)

M21 – Húngaro com reserva

Ideia. Quando as ameaças chegam em ondas, não gastar todos os interceptadores na primeira.

Origem. Lição do WTA dinâmico [3].

Como funciona. Resolve o problema de atribuição (13) como M7, mas aplica uma regra de reserva: um interceptador da base só é lançado contra uma pista de baixa prioridade ($w_j < w_P$) se, depois do lançamento, restarem mais de 20 % dos interceptadores ociosos. Pistas dentro de zonas prioritárias podem usar a reserva.

Adaptações. É uma política de horizonte deslizante: aproxima o valor de guardar recursos sem modelar explicitamente as ondas futuras, que o defensor não conhece.

Parâmetros. Reserva de 20 % da frota.

O que esperar. Pode perder ameaças de baixa prioridade na primeira onda para ganhar nas seguintes.

***Shoot–look–shoot* e salvas.** Todos os métodos permitem trocar de alvo em voo: um interceptador que perde a pista ou percebe que ela era falsa é redesignado em vez de desperdiçado. Isso é a forma *shoot–look–shoot* do WTA dinâmico [3]: observar antes de reengajar. A alternativa, *shoot–shoot*, manda dois interceptadores à mesma pista de uma vez ($\bar{n} = 2$), trocando interceptadores por chance de sucesso. Medimos as duas por ablação (Seção 8.5): desligando a troca de alvo, e permitindo salvas no M13 e no M17.

4.10 Arquiteturas comportamentais (M22, M23)

Nas árvores de comportamento, cada interceptador executa a sua própria árvore de forma independente, com o quadro de radar comum; quando dois escolhem a mesma pista, prevalece o primeiro numa ordem fixa. Diferentemente dos métodos conjuntos, a árvore decide *se* lança, além de *contra quem* — o que lhe permite poupar interceptadores.

M22 – Árvore com alocação por distância

Ideia. Engajar só ameaças que já estão perto, e sempre a mais próxima.

Origem. Ricardo Jr et al. [12].

Como funciona. A árvore tem um seletor com dois ramos, tentados em ordem: *engajar*, se existe ameaça livre e plausível a menos de R_e do interceptador (o alvo é a mais próxima); senão, *esperar*. Em voo, o interceptador mantém o alvo até perdê-lo.

Adaptações. No original, os defensores protegem um drone líder numa arquitetura líder–seguidor; aqui cada interceptador decide sozinho a partir da sua base.

Parâmetros. Raio de engajamento $R_e = 4$ km.

O que esperar. Economiza interceptadores, mas, ao esperar que a ameaça se aproxime, deixa pouco tempo para alcançar alvos rápidos.

M23 – Árvore com utilidade afim

Ideia. Decidir se vale a pena avançar sobre uma ameaça por uma nota de *utilidade* que pesa o custo de ir, o perigo da ameaça e a pressão total sobre a defesa.

Origem. Feitosa, Ricardo Jr e Santos [13].

Como funciona. A árvore original tem três ramos: *responder* (inspecionar a ameaça e então avançar ou pedir reforço), *reforço* (atender pedidos de apoio de outros defensores) e *ocioso*. A decisão de avançar é dada por uma utilidade afim — uma soma ponderada de fatores mais uma constante. Na nossa versão, o interceptador toma como alvo a ameaça livre de menor custo c_{ij} e avança se

$$U_i = c_1 \frac{c_{ij}}{100} + c_2\, \theta'_j + c_3 \frac{\Theta}{N} + c_4 > 0, \qquad \theta'_j = \frac{w_j}{w_P} e^{-t_j^Z/120}, \quad \Theta = \sum_k \theta'_k. \tag{20}$$

Os quatro termos correspondem aos da utilidade original: o tempo de deslocamento (normalizado por 100 s), a ameaça do alvo θ'_j (que vale até 1 e decai com o tempo até a zona, em escala de 2 minutos), a ameaça composta Θ (a soma das ameaças de todas as pistas,

dividida pelo número de aeronaves, que mede a pressão sobre a defesa) e uma constante que representa a aversão a risco. Os coeficientes $c_1, \ldots, c_4$ definem o comportamento: um c_1 negativo desestimula perseguir alvos distantes; c_2 e c_3 positivos estimulam reagir a ameaças fortes ou a muita pressão.

Adaptações. O ramo de reforço trata alvos que resistem a vários disparos e não se aplica a interceptadores que neutralizam com um único contato. A ameaça de uma pista usa o tempo até a zona prioritária. Como no original, os coeficientes são ajustados por um algoritmo genético; aqui, para minimizar o dano (2) médio em cenários de treino.

Parâmetros. Algoritmo genético com população de 12, 8 gerações, torneio, cruzamento aritmético (média ponderada dos pais) e mutação gaussiana de 10 % da faixa de cada coeficiente, avaliado em 10 cenários de treino.

4.11 Políticas aprendidas (M24–M29)

Ideia geral. Uma política aprendida é uma rede neural que recebe a situação vista por um interceptador e devolve uma distribuição de probabilidade sobre as ações possíveis. Em princípio, ela pode capturar regularidades que as regras ignoram — por exemplo, que certas pistas quase sempre são falsas, ou que vale esperar uma pista se confirmar antes de lançar. Todos os interceptadores consultam a mesma rede, cada um com a sua observação (*política compartilhada*).

Observação e ação. Para o interceptador da vez, a rede recebe até $K = 64$ pistas livres. Cada pista é descrita por 17 *traços de decisão* — posição e velocidade relativas ao interceptador, tempo previsto de interceptação, se é alcançável com a autonomia restante, idade, incerteza, se já está coberta por outro interceptador, altura, razão de subida, distância e velocidade de aproximação à zona prioritária, se é o alvo atual — e por 10 *traços de legitimidade*, só de radar, que ajudam a separar aeronaves de falsos alarmes: velocidade, razão de subida, altura, incerteza, idade, tempo de vida, número e taxa de medidas e distância ao radar mais próximo. Há ainda 7 *traços globais*: quantos interceptadores estão pendentes e livres, quantas pistas há, o instante, a autonomia restante, se o interceptador está em voo e a idade da pista que persegue. A ação é uma das pistas ou a ação nula.

Classificador de legitimidade. Em vez do filtro (E1), a rede tem um pequeno classificador λ que recebe os traços de legitimidade de uma pista e devolve a probabilidade de ela ser uma aeronave. Ele é treinado com o rótulo verdadeiro, que o simulador conhece (mas que nunca é dado à rede na decisão), por *entropia cruzada binária* — a perda padrão de classificação, que penaliza atribuir probabilidade baixa à classe correta —, com as classes balanceadas, porque os falsos alarmes são maioria. A probabilidade estimada entra na nota de cada pista.

Arquitetura DeepSets

Origem. Zaheer et al. [32].

Como funciona. O número de pistas varia e a ordem em que são listadas é arbitrária; a decisão não pode depender dela. A arquitetura DeepSets resolve isso processando cada pista

com a mesma rede ϕ (um *perceptron multicamadas*, MLP, a rede neural mais simples) e combinando os resultados por uma média, operação que não depende da ordem:

$$\boldsymbol{h}_j = \phi([\boldsymbol{o}_j, \boldsymbol{l}_j, \boldsymbol{g}]), \quad \bar{\boldsymbol{h}} = \tfrac{1}{|\mathcal{V}|}\textstyle\sum_j \boldsymbol{h}_j, \quad z_j = \psi([\boldsymbol{h}_j, \bar{\boldsymbol{h}}, \sigma(\lambda(\boldsymbol{l}_j))]), \quad z_\varnothing = \chi([\bar{\boldsymbol{h}}, \boldsymbol{g}]), \tag{21}$$

em que $\boldsymbol{o}_j$, $\boldsymbol{l}_j$ e $\boldsymbol{g}$ são os traços de decisão, de legitimidade e globais; $\boldsymbol{h}_j$ é o *código* da pista; $\bar{\boldsymbol{h}}$, o *contexto*, resume todas as pistas; z_j é a nota da pista, que depende do seu código, do contexto e da probabilidade de ser legítima; e $z_\varnothing$ é a nota da ação nula. As notas são convertidas em probabilidades pela função *softmax*, $\pi_j = e^{z_j} / \sum_k e^{z_k}$.

Parâmetros. MLPs de 96 unidades; classificador com camadas de 10, 64, 64 e 1 unidades.

Arquitetura com atenção

Origem. Vaswani et al. [33]; aplicação a problemas combinatórios por Kool et al. [34]; agregação por atenção em grafos em Jia et al. [18].

Como funciona. Na média do DeepSets, todas as pistas pesam igual no contexto. Com *atenção*, cada pista "consulta" as demais e decide quanto cada uma importa para ela. Cada código $\boldsymbol{h}_j$ gera uma consulta $\boldsymbol{q}_j$, uma chave $\boldsymbol{k}_j$ e um valor $\boldsymbol{v}_j$ (projeções lineares aprendidas); o peso que a pista j dá à pista k é proporcional a $e^{\boldsymbol{q}_j \cdot \boldsymbol{k}_k / \sqrt{d}}$, e o novo código de j é a média dos valores ponderada por esses pesos:

$$\text{Atenção}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d}}\Big)V. \tag{22}$$

Assim, o código de uma pista passa a refletir, por exemplo, se há outra pista mais fácil perto dela. Usamos duas camadas de *codificador Transformer* (atenção seguida de um MLP, com conexões residuais e normalização), com 4 *cabeças* de atenção em paralelo, antes da média; o restante é igual ao DeepSets. A operação continua independente da ordem das pistas.

Parâmetros. 2 camadas, 4 cabeças, dimensão 96.

M24, M25 – Imitação (DAgger)

Ideia. Aprender a decidir como um especialista, mas mais rápido, a partir de exemplos rotulados por ele.

Origem. Ross, Gordon e Bagnell [30].

Como funciona. O especialista é o húngaro ponderado (M7). Uma imitação ingênua treinaria a rede só com situações produzidas pelo especialista; mas, ao ser usada, a rede comete pequenos erros que a levam a situações que o especialista nunca produziu, e nessas ela erra mais — um efeito chamado *mudança de distribuição*. O DAgger corrige isso iterativamente: na primeira iteração, o especialista conduz os episódios e rotula as decisões; nas seguintes, a *rede* conduz, e o especialista apenas rotula a decisão certa em cada situação que ela visitou. Os exemplos de todas as iterações são acumulados, e a rede é retreinada para prever a ação do especialista, minimizando a entropia cruzada entre a sua distribuição

e a ação rotulada.

Adaptações. Quase todas as decisões do especialista são "esperar", porque a cada rodada todos os interceptadores ociosos nas bases decidem e a maioria espera. Treinada com essa proporção, uma versão preliminar da rede aprendeu a esperar sempre. Guardamos então todas as decisões de lançar ou trocar e só 10 % das de esperar, sorteadas. Para não enviesar a rede no sentido oposto, cada exemplo de esperar recebe *peso de importância* 10 (o inverso da fração guardada), de modo que a perda média é a mesma que se teria com todos os exemplos:

$$\mathcal{L}_{\mathrm{IL}} = \frac{\sum_k \omega_k \,\mathrm{CE}(\pi(\cdot \mid o_k), a_k^\star)}{\sum_k \omega_k} + 0{,}5\,\mathcal{L}_{\mathrm{leg}}, \qquad \omega_k = \begin{cases} 10, & a_k^\star = \text{esperar}, \\ 1, & \text{caso contrário}, \end{cases} \tag{23}$$

em que CE é a entropia cruzada, $a_k^\star$ a ação do especialista no exemplo k e $\mathcal{L}_{\mathrm{leg}}$ a perda do classificador de legitimidade.

Parâmetros. 5 iterações de 28 episódios, 8 épocas de treino por iteração, otimizador Adam com taxa 10^{-3}, 3 sementes de treino.

O que esperar. A rede pode, no melhor caso, igualar o especialista; a vantagem é a velocidade de decisão, e o interesse é servir de ponto de partida para o reforço.

M26–M29 – Reforço (PPO)

Ideia. Aprender diretamente a minimizar o dano, por tentativa e erro, sem depender de um especialista.

Origem. Otimização de política proximal (PPO) [31]; políticas centralizadas para alocação por reforço [16–18].

Como funciona. A rede joga episódios completos e recebe, a cada passo, uma *recompensa* que premia a interceptação de ameaças armadas, pune impactos pelo seu peso, cobra cada interceptador desperdiçado (inclusive os gastos em iscas) e cobra um pequeno custo pelo tempo com ameaças ainda vivas, para estimular rapidez:

$$r_t = \sum_{a:\,\nu_a>0} \mathbf{1}\{a \text{ interceptada}\} - \sum_a w_a \mathbf{1}\{a \text{ atinge o impacto}\} - \eta\,\Delta W_t - \frac{N_{\mathrm{vivas}}(t)\,\Delta t}{300\,N}, \tag{24}$$

em que ΔW_t é o número de interceptadores desperdiçados no passo. Os dois primeiros termos e o terceiro correspondem a (2) e (3): a recompensa ensina a proteger sem disparar à toa. O objetivo é maximizar a soma das recompensas. Para isso, além da política, a rede tem uma cabeça de *valor*, que estima quanto de recompensa ainda se espera obter a partir de uma situação. A *vantagem* $\hat{A}_t$ de uma ação é quanto ela foi melhor que o esperado: positiva se a recompensa obtida superou a estimativa de valor. Usamos a estimativa GAE, uma média ponderada das diferenças entre recompensa observada e valor previsto ao longo dos passos seguintes, que equilibra ruído e viés. A atualização torna mais prováveis as ações de vantagem positiva e menos as de vantagem negativa. O PPO limita o tamanho de cada

Tabela 7. Objetivo, garantia e custo por rodada. n: interceptadores da rodada; K: pistas; G: iterações; P: população; R: rodadas de consenso.

Métodos	Objetivo	Garantia	Custo por rodada
M2–M4	tempo (individual)	nenhuma; depende da ordem da fila	$O(nK)$
M5	ameaça, depois tempo	nenhuma	$O(K \log K + nK)$
M6–M8	tempo total (13)	ótimo da rodada	$O(n^2K)$
M9	tempo total (13)	a $n\varepsilon$ do ótimo	pseudo-polinomial
M10–M11	lances w_j/c_{ij}	$\geq 50\,\%$ do ótimo [29]	$O(R\,n^2K)$
M12	idem, comunicação local	sem garantia de ausência de conflito	$O(3\,n^2K)$
M13	valor esperado (10)	ótimo para armas idênticas	$O(n^2K)$
M14, M15	valor esperado, $\bar{n} = 1$	ótimo da rodada	$O(n^2K)$ / exponencial no pior caso
M16	valor esperado	ótimo local, $\geq$ M13	$O(G(nK + n^2)nK)$
M17–M20	valor esperado	$\geq$ M13	$O(GP\,nK)$
M21	tempo + reserva	nenhuma	$O(n^2K)$
M22, M23	regra / utilidade	nenhuma	$O(K)$ por interceptador
M24–M29	imitação / recompensa	nenhuma	uma inferência por interceptador

atualização para não destruir o que já foi aprendido:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}_t\Big[\min\left(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\Big], \qquad \rho_t = \frac{\pi_{\text{nova}}(a_t \mid o_t)}{\pi_{\text{antiga}}(a_t \mid o_t)}, \tag{25}$$

em que ρ_t mede quanto a nova política mudou a probabilidade da ação tomada; o recorte (clip) ignora mudanças maiores que $\pm\epsilon$. Somam-se uma perda de valor, um bônus de *entropia* (que estimula a política a continuar explorando ações diferentes) e a perda de legitimidade.

Adaptações. M26 e M27 partem das redes de imitação (M24, M25), com taxa de aprendizado menor, para refinar o comportamento já aprendido; M28 e M29 partem do zero. A perda de legitimidade é mantida como tarefa auxiliar.

Parâmetros. 40 atualizações de 7 episódios cada, 4 passadas pelos dados por atualização, lotes de 512 exemplos, $\epsilon = 0{,}2$, fator de desconto $\gamma = 0{,}995$, parâmetro do GAE 0,95, bônus de entropia 0,01; taxa de aprendizado 10^{-4} (M26, M27) ou 3×10^{-4} (M28, M29); 3 sementes de treino, só nos cenários de treino.

O que esperar. O reforço do zero é o mais custoso de treinar e o mais variável entre sementes; o refino a partir da imitação tende a ser mais estável.

4.12 Resumo de custo e garantias

A Tabela 7 resume, para cada família, o objetivo otimizado, a garantia conhecida e o custo por rodada.

5 Ambiente de simulação

Todos os métodos são avaliados no mesmo simulador, escrito em Python e aberto. Esta seção descreve o que ele modela e por quê; a Tabela 8 lista os parâmetros. O simulador avança em passos de 1 s sobre um modelo digital de elevação real (Copernicus GLO-30, com resolução de

30 m), e usa subpassos de 0,1 s quando algum interceptador está perto de uma aeronave, para que o contato seja verificado com precisão.

5.1 Radares

Um radar emite pulsos e mede o eco refletido pelos objetos; a intensidade do eco em relação ao ruído do receptor é a **relação sinal–ruído** (SNR), e quanto maior ela, mais provável detectar o alvo e mais precisa a medida. O SNR segue a *equação do radar*,

$$\mathrm{SNR} = \mathrm{SNR}_{50}\Big(\frac{R_0}{r}\Big)^4 \sigma_{\mathrm{ef}}\, 10^{-L/10}, \tag{26}$$

em que r é a distância, R_0 é o alcance em que um alvo de 1 m^2 é detectado com 50 % de probabilidade, σ_{ef} é a **seção reta radar** (o "tamanho" do alvo visto pelo radar, em m^2) e L são as perdas em decibéis. O expoente 4 vem de o sinal percorrer a distância duas vezes, ida e volta. A seção radar depende do ângulo de visada: um avião visto de lado reflete mais que visto de frente.

A detecção segue o modelo de alvo **Swerling I** [22], que representa alvos cuja seção radar flutua de uma varredura para outra (por pequenas mudanças de atitude), com probabilidade de detecção $P_d = P_{fa}^{1/(1+\mathrm{SNR})}$, em que $P_{fa} = 10^{-6}$ é a probabilidade de falso alarme por célula de resolução. As perdas L incluem a atenuação atmosférica e a **difração** no relevo: quando um morro fica entre o radar e o alvo, o sinal não é cortado de forma abrupta, mas se curva por cima do obstáculo com uma atenuação que cresce à medida que o alvo desce na sombra do morro, calculada pelo modelo de gume de faca. A curvatura da Terra também é considerada.

Cada radar gira com período de 4 s, de modo que mede cada alvo apenas quando o feixe passa por ele, e os radares não são sincronizados entre si. O ruído das medidas de distância e de ângulo cresce quando o SNR cai. Cada radar tem ainda **vieses** — erros fixos de distância e de azimute, como os causados por um desalinhamento da antena — que o rastreador desconhece, e produz falsos alarmes concentrados perto da antena, onde os ecos do terreno são mais fortes.

5.2 Rastreador

As medidas de todos os radares são combinadas por um **rastreador**, que mantém a lista de pistas. Para cada pista, ele estima posição e velocidade com um **filtro de Kalman**, que combina a previsão do movimento com as novas medidas, pesando cada uma pela sua incerteza. Como as ameaças alternam voo reto, órbitas e manobras, usamos um filtro **IMM** (*interacting multiple models*) [24], que roda dois filtros em paralelo — um supondo movimento quase uniforme e outro admitindo acelerações maiores — e combina as estimativas segundo qual modelo explica melhor as medidas recentes.

A cada varredura, o rastreador decide qual medida pertence a qual pista (a **associação de dados**). Para isso, mede a distância entre a medida e a posição prevista de cada pista em unidades de incerteza (a *distância de Mahalanobis*), descarta pares improváveis e escolhe a associação global de menor custo [40]. Medidas não associadas iniciam pistas novas. Uma pista só é **confirmada**, e passada aos métodos de alocação, após 3 medidas; é descartada após 20 s sem atualização. Pistas duplicadas (o mesmo alvo visto por dois radares) são fundidas.

5.3 Ameaças

Cada ameaça percorre uma rota suavizada $\boldsymbol{\gamma}(s)$, em que s é a distância percorrida, a uma velocidade constante sorteada na faixa da sua classe, a uma altura sobre o terreno que varia entre os pontos de controle da rota e com um **mergulho** final até o ponto de impacto, cuja inclinação depende da classe (Tabela 9). As classes e os seus parâmetros vêm de fontes abertas sobre os vetores empregados na guerra na Ucrânia (Seção 6); a Tabela 9 indica quais valores são hipóteses do modelo.

Os drones FPV, pilotados, podem manobrar. Como no gerador de manobras do simulador, a manobra é um **deslocamento lateral** $y(t)$ em relação à rota, medido na direção perpendicular a ela, $\boldsymbol{n}(s)$:

$$\boldsymbol{p}(t) = \boldsymbol{\gamma}(s) + y\,\boldsymbol{n}(s), \qquad \ddot{y} = \mathrm{sat}_{a_e}(\omega^2(y_{\mathrm{ref}} - y) - 2\zeta\omega\dot{y}). \tag{27}$$

A segunda equação faz o deslocamento seguir uma referência y_{ref} como um sistema massa–mola amortecido, com frequência ω e amortecimento ζ; a função sat_{a_e} limita a aceleração lateral a a_e, o que representa a agilidade máxima da aeronave. O deslocamento se anula perto do impacto, de modo que o ponto de queda não muda com a manobra. Nos FPV, a referência é sorteada a intervalos aleatórios (curvas aleatórias); em metade deles, há também uma guinada brusca para o lado oposto a um interceptador que se aproxima (**quebra reativa**), como faria um piloto que vê o interceptador. As demais classes seguem a rota programada sem manobrar.

As iscas voam como as demais ameaças, com assinatura de radar parecida ou maior, mas têm valor de dano nulo: o impacto de uma isca não conta no dano, e o interceptador que a abate é contado como desperdiçado.

5.4 Interceptadores

Os interceptadores voam com velocidade constante e com uma taxa máxima de curva, que representa a sua aceleração lateral limitada. Em trânsito, voam em direção ao ponto previsto de encontro da última estimativa da pista (6). Quando uma ameaça entra no alcance do sensor próprio, passam à guiagem terminal, com uma lei fixa escolhida pelo experimento da Seção 8.2. O contato é verificado continuamente: em cada subpasso, calcula-se a menor distância entre o interceptador e cada ameaça, supondo movimento retilíneo durante o subpasso, e há contato se ela for menor que r_c. A velocidade de 65 m/s (234 km/h) segue a de interceptadores empregados contra drones de ataque, que anunciam cruzeiro de 140 a 170 km/h e máximo de 280 km/h [41, 42]; ela precisa superar a do drone de ataque, de até 51 m/s, para que a interceptação pela cauda seja possível.

5.5 Rodadas de decisão e fim do episódio

A cada 2 s ocorre uma rodada de decisão. Decidem, nesta ordem, os interceptadores em voo cuja reavaliação venceu — a cada 10 s, ou imediatamente se a pista que perseguiam desapareceu — e os interceptadores ociosos nas bases. Decidir primeiro os que já estão em voo evita lançar um interceptador novo contra uma pista que um em voo poderia assumir.

O episódio termina quando todas as ameaças foram interceptadas ou atingiram o impacto, ou quando acaba a duração da missão. Se a defesa fica sem interceptadores — nenhum na base

Tabela 8. Parâmetros do ambiente de simulação. Os parâmetros de cada classe de ameaça estão na Tabela 9.

Radares (4–7 por missão, em topos de morro)	
alcance ($P_d = 50\%$, 1 m^2)	4 km; antena a 15 m do solo; varredura de 4 s; 3 GHz
ruído / vieses	$\sigma_r = 40$ m em SNR de referência; vieses fixos por radar
falsos alarmes	2–5 por varredura
Ameaças (12–48 por missão, em 4 classes)	
velocidade / altura	17–51 m/s; 20–1500 m sobre o terreno, conforme a classe
seção radar	0,02–1,5 m^2 conforme a classe, dependente do aspecto
manobra (FPV)	$a_e \leq 8$ m/s^2; $\omega = 0{,}7$ rad/s, $\zeta = 0{,}8$; quebra reativa a 300 m
Interceptadores ($\lceil 1{,}25N \rceil$, em 2–4 bases)	
velocidade / autonomia	65 m/s; 420 s
aceleração lateral	30 m/s^2
sensor próprio / contato	300 m / 3 m
Simulação	
passo / subpasso	1 s / 0,1 s
rodada de decisão / reavaliação	2 s / 10 s
penalidade por desperdício η	0,2

nem em voo —, ninguém mais pode deter as ameaças restantes, e todas elas, inclusive as que ainda não entraram no mapa, são contadas como impacto. No fim, os interceptadores ainda em voo, que não neutralizaram nada, contam como desperdiçados.

6 Conjunto de missões

Um benchmark de alocação só é útil se as suas missões se parecerem com ataques reais. Os cenários da literatura costumam ser sintéticos: alvos e rotas sorteados no mapa e, muitas vezes, radares posicionados para cobrir as rotas do próprio ataque — o que dá ao defensor um conhecimento que ele não teria. Construímos as missões a partir de três fontes de realismo: *onde* se ataca (ativos reais de infraestrutura sobre o relevo real), *com o quê* se ataca (classes de ameaça com parâmetros de fontes abertas) e *como* se ataca (eixo de ameaça, rotas que aproveitam o relevo, iscas e chegada coordenada). A defesa, por sua vez, é montada sem conhecer o ataque.

6.1 Cidades e ativos

As missões cobrem seis cidades: São José dos Campos, Taubaté e Itajubá, polos aeroespacial, industrial e de energia do Vale do Paraíba e do sul de Minas Gerais, e as três maiores concentrações de infraestrutura e de governo do país, São Paulo, Rio de Janeiro e Brasília. Cada mapa tem 12 a 14 km de lado, sobre o modelo de elevação Copernicus GLO-30 [43]; os relevos vão do planalto quase plano de Brasília às serras do Vale e aos maciços do Rio, o que produz desde regiões de boa cobertura de radar até sombras extensas atrás dos morros.

Os **ativos** — os alvos possíveis — são extraídos do OpenStreetMap [44]: usinas e aeródromos (importância 4), subestações, estações de tratamento de água e esgoto e áreas industriais de mais de 50 ha (importância 3), torres de telecomunicação e áreas industriais de 5 a 50 ha (importância 2). Entre 31 (Itajubá) e 329 (São Paulo) ativos por cidade. A extração é gravada junto com o

código, de modo que as missões não mudam se o mapa colaborativo mudar.

Em cada missão, de dois a três ativos são **defendidos**: viram zonas prioritárias, com raio igual ao do ativo mais 250 m (entre 300 e 1000 m). Eles são sorteados em dois passos — primeiro a categoria, com peso proporcional ao cubo da importância, depois o ativo dentro dela, favorecendo os maiores —, para que as dezenas de torres de uma cidade não dominem a escolha só por serem numerosas. Nas 192 missões, os ativos defendidos são estações de água (109), subestações (99), aeródromos (90), áreas industriais (80), usinas (74) e torres de telecomunicação (28).

6.2 Defesa

A defesa é montada a partir dos ativos defendidos, *sem conhecer o ataque*. De quatro a sete radares são postos nos topos de morro que mais cobrem a área até 5 km de cada ativo defendido, escolhidos de forma gulosa, com sorteio entre os três melhores a cada passo. Há uma base de interceptadores a 0,8–2 km de cada ativo defendido e, às vezes, uma base central, com $\lceil 1{,}25N \rceil$ interceptadores no total — uma folga de 25 % sobre o número de ameaças, que precisa cobrir também os interceptadores gastos com falsos alarmes e iscas. Ao contrário de um gerador que posiciona os radares para cobrir as rotas do ataque, nenhuma missão é descartada por cobertura: a fração das rotas coberta pelos radares, a 120 m sobre o terreno, varia de 38 a 100 % e é uma característica da missão, não um critério de seleção.

6.3 Classes de ameaça

A Tabela 9 resume as quatro classes. As velocidades e alturas vêm de fontes abertas sobre os vetores empregados na guerra na Ucrânia: o drone de ataque de longo alcance do tipo Shahed-136/Geran-2, que voa a 150–185 km/h numa rota pré-programada e usa tanto alturas muito baixas, para se esconder no relevo, quanto alturas de vários quilômetros [45–47]; a isca do tipo Gerbera, mais barata, de assinatura de radar parecida ou ampliada, lançada junto com os drones de ataque para atrair a defesa — numa proporção relatada de cerca de 40 % da onda [48–50]; a munição vagante do tipo Lancet-3, de 80–110 km/h, que orbita sobre o alvo e mergulha a cerca de 300 km/h [51, 52]; e o drone FPV de ataque, que em combate voa a 60–80 km/h, com máximo de cerca de 120 km/h [53]. A seção radar, o valor de dano e a inclinação do mergulho são hipóteses do modelo; o valor de dano é aproximadamente proporcional, de forma comprimida, à carga explosiva.

Toda missão mistura as quatro classes, em uma de quatro **composições** (Tabela 10): de *saturação*, que segue a proporção relatada de iscas; de *longo alcance*, dominada por drones de ataque; *tática*, dominada por munições vagantes e FPV; e *equilibrada*. Cada classe tem pelo menos uma ameaça.

6.4 Ataque

Eixo de ameaça. Um ataque real vem de uma direção, a do território de onde é lançado. Cada missão sorteia um **eixo de ameaça** e um setor de 60 a 90°; as ameaças entram pela borda do mapa dentro desse setor, que pode girar até 20° entre ondas. Como os mapas têm 12 a 14 km, só o trecho final das rotas de longo alcance é simulado.

Tabela 9. Classes de ameaça. Velocidade e altura de fontes abertas (ver texto); seção radar (RCS), valor de dano ν_a e mergulho são hipóteses do modelo. Mergulho: inclinação da rampa final até o impacto.

Classe	Velocidade	Altura sobre o terreno	Mergulho	RCS (m^2)	ν_a	Comportamento
ataque	42–51 m/s	60–150 m (50 %) ou 500–1500 m	45°	0,3–1,0	1	rota pré-programada
isca	27–44 m/s	80–200 m (70 %) ou 500–1200 m	raso	0,5–1,5	0	como o ataque; chega antes
vagante	22–31 m/s	300–800 m	63°	0,1–0,4	0,5	orbita o alvo (400–700 m) e mergulha
FPV	17–33 m/s	20–50 m	raso	0,02–0,1	0,25	curvas aleatórias; metade reage ao interceptador

Tabela 10. Composições do enxame: fração das ameaças em cada classe.

Composição	ataque	isca	vagante	FPV
saturação	45 %	40 %	10 %	5 %
longo alcance	60 %	25 %	10 %	5 %
tática	15 %	10 %	40 %	35 %
equilibrada	25 %	25 %	25 %	25 %

Alvos. Três quartos das ameaças armadas vão a um ativo defendido; as demais, a ativos não defendidos, cujo impacto tem penalidade baixa (w_O). As iscas vão a ativos quaisquer. O ponto de impacto é sorteado dentro da área do ativo.

Rotas sobre o relevo. As rotas são caminhos de menor custo numa grade de 150 m, do ponto de entrada ao alvo, com custo por célula $1 + \beta\,\tilde{h} + 0{,}6\,\xi$, em que $\tilde{h}$ é a altura do terreno normalizada no mapa, ξ um ruído suave sorteado a cada missão (que torna as rotas diferentes entre missões) e β depende da altura de voo: $\beta = 3$ para quem voa baixo, que passa a seguir os vales e a se esconder do radar atrás dos morros, e $\beta = 0{,}3$ para quem voa alto, cuja rota é quase reta. Uma penalidade perto da borda evita rotas que correm rente a ela. A rota é simplificada a um ponto de controle a cada 700 m e suavizada. As munições vagantes terminam com uma órbita completa sobre o alvo antes do mergulho.

Chegada coordenada. As ameaças entram em uma a três **ondas**, separadas por 120 a 300 s. Dentro de cada onda, o instante de entrada de cada ameaça é calculado a partir do comprimento da sua rota e da sua velocidade para que todas cheguem ao alvo num intervalo de 60 s — de modo que vetores lentos e rápidos, vindos de pontos diferentes, saturem a defesa ao mesmo tempo. As iscas chegam 30 a 90 s antes, para atrair os interceptadores.

6.5 Aleatoriedade e delineamento

Cada missão é um *gerador* de episódios, e não uma situação fixa: a cada execução, os pontos de controle das rotas (num raio de 100 a 250 m), as velocidades, as alturas, os pontos de entrada e os parâmetros das manobras são sorteados de novo dentro das faixas da classe. Isso evita que um método aprendido decore situações específicas.

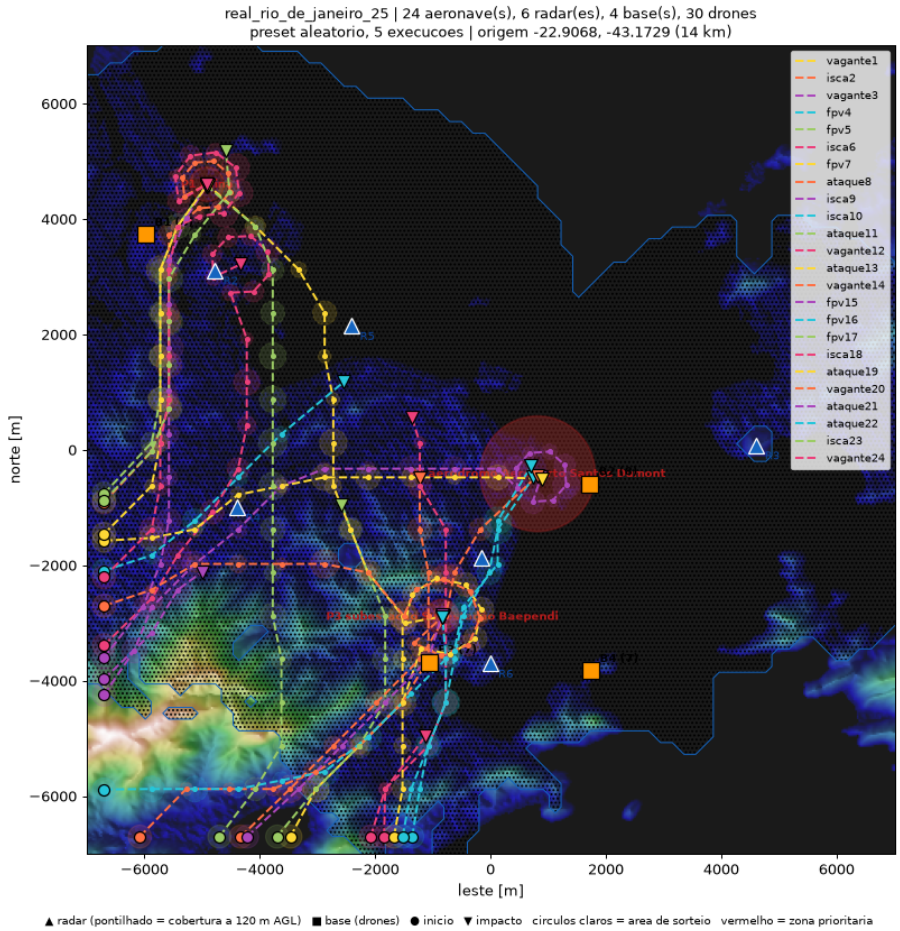


Figura 1. Missão de teste no Rio de Janeiro com composição equilibrada: 24 ameaças (6 de cada classe) entrando pelo setor sudoeste, em rotas tracejadas que seguem o relevo e terminam nos impactos (▼); as munições vagantes orbitam os alvos antes de mergulhar. Três ativos defendidos (vermelho), entre eles o aeroporto Santos Dumont; seis radares nos morros (▲, com a cobertura a 120 m sobre o terreno em pontilhado) e quatro bases com 30 interceptadores (■).

O **delineamento** é *balanceado*: em cada cidade, cada combinação de tamanho do enxame, $N \in \{12, 24, 36, 48\}$, e composição aparece exatamente duas vezes, o que dá 32 missões por cidade e 192 no total, com 5760 ameaças (2004 drones de ataque, 1440 iscas, 1260 munições vagantes e 1056 FPV). Assim, tamanho, composição e cidade não ficam confundidos e o efeito de cada fator pode ser medido separadamente. A semente de cada cidade é derivada do seu nome, de modo que acrescentar ou retirar cidades não muda as missões das demais.

6.6 Treino e teste

As 64 missões do Rio de Janeiro e de Brasília formam o conjunto de **teste**: os métodos com parâmetros ajustados a partir de dados (a árvore com utilidade e as redes neurais) usam só as 128 missões das outras quatro cidades, e são avaliados em cidades que nunca viram — com outro relevo, outros ativos e outro arranjo da defesa. Brasília, plana, e o Rio, entre o mar e os maciços, são também os dois relevos mais diferentes dos do treino. A Figura 1 mostra uma missão de teste.

7 Protocolo experimental

Sementes pareadas. Uma **semente** é o número que inicializa o gerador de números aleatórios; com a mesma semente, os mesmos sorteios se repetem. A execução k de cada missão usa a mesma semente para todos os métodos, de modo que todos enfrentam exatamente as mesmas rotas, velocidades, manobras, medidas de radar, falsos alarmes e iscas. As diferenças entre métodos, portanto, não vêm de sorteios diferentes, e a comparação pode ser feita par a par.

O que é avaliado. Os métodos sem parâmetros ajustados a partir de dados (M1–M22) são avaliados nas 192 missões, com 5 execuções cada (960 episódios por método). A árvore com utilidade (M23) é ajustada nas missões de treino e avaliada em todas. As redes (M24–M29) são treinadas nas 128 missões de treino e avaliadas nas 64 de teste (320 episódios), para cada uma das 3 sementes de treino. As ablações usam 3 execuções por missão.

Métricas. A métrica principal é o dano normalizado D (2); reportamos também o custo C (3), que soma ao dano a penalidade dos interceptadores desperdiçados, a fração das ameaças armadas interceptadas, o número de impactos em zonas prioritárias, os interceptadores desperdiçados e gastos por ameaça e o tempo de decisão.

Intervalos de confiança. Reportamos a média de cada métrica com um **intervalo de confiança** de 95 %, a faixa que, com 95 % de confiança, contém o valor médio verdadeiro. Ele é obtido por ***bootstrap***: reamostra-se o conjunto de dados com reposição milhares de vezes, recalcula-se a média em cada reamostra e tomam-se os percentis de 2,5 e 97,5 %. Reamostramos *missões*, e não episódios, porque episódios da mesma missão são parecidos entre si; tratá-los como independentes produziria intervalos estreitos demais.

Testes de significância. Para decidir se um método é melhor que a referência (o húngaro ponderado, M7), usamos o **teste de Wilcoxon pareado**: para cada par (missão, semente), calcula-se a diferença de dano entre os dois métodos, e o teste verifica se essas diferenças tendem a um lado mais do que o acaso explicaria, sem supor que sigam uma distribuição normal. Reportamos o valor p: a probabilidade de observar diferenças tão sistemáticas se os dois métodos fossem, na verdade, equivalentes. Para as redes, cada par usa a média sobre as sementes de treino.

Custo computacional. O tempo de decisão é medido por chamada, com um processo por episódio. [**A FAZER:** processador da máquina em que o benchmark rodar.]

8 Resultados

8.1 Validação das implementações

Antes de comparar desempenho, é preciso confirmar que as implementações estão corretas. Uma forma objetiva de fazê-lo é usar as equivalências que a teoria garante: certos pares de algoritmos,

Tabela 11. Validação das implementações e custo por rodada de decisão: 320 rodadas de 8 cenários, resolvidas por todos os algoritmos (instância mediana com 49 pares interceptador–pista; máxima com 376). Mesma designação: fração dos interceptadores designados igual à referência teoricamente equivalente; mesmo valor: fração das rodadas em que o valor do objetivo coincide (a designação pode diferir quando há ótimos alternativos). Objetivo relativo: valor do objetivo (10) dividido pelo melhor encontrado na rodada.

Algoritmo	Referência equivalente	Mesma designação (%)	Mesmo valor (%)	Objetivo relativo	Temp
Húngaro ponderado	–	–	–	–	
Leilão (Bertsekas)	Húngaro ponderado	99,6	100,0	–	
Guloso global	–	–	–	–	
CBAA distribuído	Guloso global	100,0	100,0	–	
Húngaro sobre WTA	–	–	–	1,000	
MILP (HiGHS)	Húngaro sobre WTA	96,8	100,0	–	
MMR	–	–	–	0,999	
Busca local (VLSN)	–	–	–	1,000	
Algoritmo genético	–	–	–	1,000	
Colônia de formigas	–	–	–	1,000	
Recozimento simulado	–	–	–	0,999	
Enxame de partículas	–	–	–	0,999	

embora funcionem de modo diferente, devem chegar à mesma solução. Em 320 rodadas de decisão de 8 cenários, todos os algoritmos resolveram exatamente a mesma instância (Tabela 11).

Os três pares teoricamente equivalentes — leilão e húngaro ponderado (M9 e M7, mesmo problema de atribuição), CBAA com grafo completo e guloso global (M11 e M10) e MILP e húngaro sobre o WTA (M15 e M14, mesmo problema linear) — atingem o mesmo valor do objetivo em 100 % das rodadas. A designação coincide em 99,6, 100 e 96,8 % dos interceptadores, respectivamente; as diferenças vêm de soluções alternativas com o mesmo valor (empates), e não de erros.

Entre os métodos que otimizam o valor esperado (10), o MMR fica em média a 0,1 % do melhor valor encontrado na rodada, mas chega a ficar 7,6 % abaixo em algumas; a busca local é a que mais se aproxima do melhor (99,99 % em média, pior caso 98,6 %). O custo varia em duas ordens de grandeza: atribuição, leilão, MMR e busca local resolvem uma rodada em 4 a 6 ms (mediana), o MILP em 16 ms, o algoritmo genético em 93 ms, o recozimento simulado em 234 ms e a colônia de formigas em 487 ms.

8.2 Escolha da guiagem terminal

A guiagem é um componente fixo do benchmark, mas precisa ser escolhida. Um experimento 1×1 — um interceptador contra uma aeronave, 300 encontros por combinação, os mesmos para todas as leis — comparou leis de guiagem contra cada perfil evasivo (Tabela 12), com dois objetivos: fixar a lei usada no benchmark e medir a probabilidade P_T de contato na primeira passagem, usada no modelo de sucesso (9). A navegação proporcional com constante $N' = 4$ teve a maior taxa de contato na primeira passagem contra as manobras não reativas, e é a lei usada; sua taxa média de contato na primeira passagem, 0,85, é o valor de P_T. A guiagem preditiva (MPC), que supõe velocidade constante do alvo ao longo do horizonte, ficou abaixo da navegação proporcional contra as manobras (por exemplo, 71 contra 88 % de contato na primeira passagem contra zigue-zague, com 30 m/s^2) e custou ordens de grandeza mais tempo por passo — um

Tabela 12. Guiagem terminal 1×1: probabilidade (%) de contato na primeira passagem / em até 60 s, para 300 encontros por célula (mesmos encontros para todas as leis). Drone a 42 m/s; aeronave a 22–32 m/s com até 10 m/s² de aceleração lateral; raio de contato 2 m; $a_{\max}$ do drone em m/s².

Manobra	Lei	$a_{\max} = 15$	$a_{\max} = 30$	$a_{\max} = 60$
nenhuma	PP	70 / 93	78 / 100	88 / 100
	PN (N'=3)	100 / 100	100 / 100	100 / 100
	PN (N'=4)	100 / 100	100 / 100	100 / 100
	APN (N'=4)	100 / 100	100 / 100	100 / 100
	MPC	71 / 78	93 / 99	96 / 99
zigue-zague	PP	32 / 53	76 / 100	86 / 100
	PN (N'=3)	75 / 95	83 / 100	83 / 100
	PN (N'=4)	87 / 99	88 / 100	88 / 100
	APN (N'=4)	72 / 92	67 / 98	66 / 99
	MPC	38 / 67	71 / 82	78 / 88
reativa	PP	80 / 98	90 / 98	98 / 99
	PN (N'=3)	80 / 100	80 / 100	80 / 100
	PN (N'=4)	85 / 100	85 / 100	85 / 100
	APN (N'=4)	67 / 78	74 / 99	74 / 99
	MPC	70 / 92	88 / 92	85 / 92
mista	PP	78 / 96	90 / 97	94 / 99
	PN (N'=3)	81 / 99	83 / 100	83 / 100
	PN (N'=4)	82 / 98	83 / 100	83 / 100
	APN (N'=4)	72 / 90	65 / 97	70 / 99
	MPC	69 / 93	88 / 95	88 / 95
curvas aleatórias	PP	55 / 80	78 / 100	85 / 100
	PN (N'=3)	91 / 98	94 / 100	94 / 100
	PN (N'=4)	94 / 99	96 / 100	96 / 100
	APN (N'=4)	88 / 96	82 / 100	84 / 100
	MPC	63 / 78	79 / 89	86 / 90

primeiro indício, no nível de um único encontro, de que a hipótese de velocidade constante é frágil diante de alvos evasivos.

8.3 Comparação dos métodos de alocação

[**A FAZER:** comentar as tabelas principais (teste e treino): ranking por dano e por custo, quem desperdiça interceptadores em iscas, significância contra M7.]

[**A FAZER:** tabela `tabelas/principal_teste` será gerada pelos scripts de análise] [**A FAZER:** tabela `tabelas/principal_treino` será gerada pelos scripts de análise]

8.4 Efeito da composição do enxame, da cidade e da escala

[**A FAZER:** comentar: quais métodos sofrem mais com a saturação por iscas (desperdício) e com a composição tática (FPV e munições vagantes, lentas e baixas); diferença entre cidades de treino e de teste; crescimento do dano com N.]

[**A FAZER:** tabela `tabelas/composicao` será gerada pelos scripts de análise] [**A FAZER:** tabela `tabelas/cidade` será gerada pelos scripts de análise] [**A FAZER:** tabela `tabelas/tamanho` será gerada pelos scripts de análise]

8.5 Ablações: troca de alvo, salvas, comunicação e guiagem

Uma **ablação** remove ou altera um componente de um método para medir quanto ele contribui. [**A FAZER:** comentar.]

[**A FAZER:** tabela `tabelas/ablacao` será gerada pelos scripts de análise]

8.6 Métodos aprendidos

[**A FAZER:** comentar imitação, PPO do zero e refinado, DeepSets × atenção, variância entre sementes, generalização.]

9 Discussão

[**A FAZER:** síntese por família; efeito das iscas e do desperdício (dano × custo); efeito da composição e do voo baixo sobre a previsão de velocidade constante; generalização para cidades nunca vistas; custo computacional e escalabilidade; recomendações.]

9.1 Limitações

- **Modelo de interceptador.** Os interceptadores são pontos de massa em duas dimensões, com velocidade constante e aceleração lateral limitada; não há dinâmica de atitude nem vento sobre eles. Por isso, a altura da ameaça afeta a detecção pelo radar, mas não o engajamento.
- **Modelo de ameaça.** A velocidade de cada ameaça é constante ao longo da rota (o aumento de velocidade da munição vagante no mergulho não é modelado), e só o trecho final das rotas de longo alcance, dentro de um mapa de 12 a 14 km, é simulado. A seção radar, o valor de dano de cada classe e a penalidade por desperdício ($\eta = 0{,}2$) são hipóteses do modelo; os parâmetros de fontes abertas vêm de reportagens e análises secundárias, não de especificações oficiais.
- **Ativos.** Os alvos dependem do mapeamento colaborativo do OpenStreetMap, cuja cobertura varia entre as cidades; a importância de cada categoria de ativo é uma escolha do modelo.
- **Sensor próprio ideal.** Na fase final, o interceptador vê a aeronave sem erro.
- **Adaptações.** As técnicas foram adaptadas a interceptadores descartáveis de contato único. A árvore com utilidade (M23) perde o ramo de reforço, e o MILP (M15) representa só a parte de alocação de [15], sem obstáculos nem bateria. Só as redes treinadas por reforço conhecem a penalidade por desperdício; os métodos de WTA usam o seu próprio custo de lançamento, e os demais não têm nenhum.
- **Modelo de sucesso.** Os métodos M13–M20 dependem de (9), calibrado num experimento 1×1; um modelo melhor poderia favorecer essa família.
- **Orçamento de treino.** As redes foram treinadas com um orçamento fixo e modesto; resultados com mais treino podem diferir.

10 Conclusão

[**A FAZER:**]

Disponibilidade de código e dados

O simulador, os geradores de missões, a extração dos ativos do OpenStreetMap, as 192 missões, as implementações dos 29 métodos (um arquivo por método), os modelos treinados e os scripts que geram as tabelas e figuras deste artigo estão disponíveis em [**A FAZER:** repositório]. Qualquer método pode ser rodado isoladamente sobre qualquer missão (`rodar.py`).

Referências

[1] Alan S. Manne. A target-assignment problem. *Operations Research*, 6(3):346–351, 1958.

[2] G. G. den Broeder, R. E. Ellison, e L. Emerling. On optimum target assignments. *Operations Research*, 7(3):322–326, 1959.

[3] Patrick A. Hosein e Michael Athans. Some analytical results for the dynamic weapon-target allocation problem. Technical Report LIDS-P-1944, MIT Laboratory for Information and Decision Systems, 1990.

[4] Stuart P. Lloyd e Hans S. Witsenhausen. Weapons allocation is NP-complete. In *Proceedings of the 1986 Summer Computer Simulation Conference*, pages 1054–1058, 1986.

[5] Huaiping Cai, Jingxu Liu, Yingwu Chen, e Hao Wang. Survey of the research on dynamic weapon-target assignment problem. *Journal of Systems Engineering and Electronics*, 17(3): 559–565, 2006.

[6] J. N. Roux e J. H. van Vuuren. Threat evaluation and weapon assignment decision support: A review of the state of the art. *ORiON*, 23(2):151–187, 2007.

[7] Alexander Kline, Darryl Ahner, e Raymond Hill. The weapon-target assignment problem. *Computers & Operations Research*, 105:226–236, 2019.

[8] Bellingcat. The poor man's air force? rebel drones attack russia's airbase in syria. https://www.bellingcat.com/news/middle-east/2018/01/12/the_poor_mans_airforce/, 2018. Acesso em out. 2026.

[9] Manohar Parrikar Institute for Defence Studies and Analyses. Saudi drone attacks: A preliminary military evaluation. https://idsa.in/publisher/comments/saudi-drone-attacks-a-preliminary-military-evaluation, 2019. Acesso em out. 2026.

[10] PBS NewsHour. Israel says it intercepted 99 percent of iran's drones and missiles. https://www.pbs.org/newshour/world/live-updates-israel-says-it-intercepted-99-percent-of-irans-drones-and-missiles, 2024. Acesso em out. 2026.

[11] Ukrainska Pravda. Ukrainian darknode unit's interceptor drones down first hundred russian shaheds. https://www.pravda.com.ua/eng/news/2025/05/19/7512894, 2025. Acesso em out. 2026.

[12] Jorge A. Ricardo Jr, Leonardo Giacomossi, João F. S. Trentin, João F. B. Brancalion, Marcos R. O. A. Maximo, e Davi A. Santos. Cooperative threat engagement using drone swarms. *IEEE Access*, 11:9529–9546, 2023.

[13] Arthur V. B. Feitosa, Jorge A. Ricardo Jr, e Davi A. Santos. Behavior tree-based decision-making for aerial threat engagement using a drone swarm. In *Proceedings of the 28th ABCM International Congress of Mechanical Engineering (COBEM 2025)*, Curitiba, PR, Brazil, 2025. COB-2025-0063.

[14] Rubens J. M. Afonso, Marcos R. O. A. Maximo, e Roberto K. H. Galvão. Task allocation and trajectory planning for multiple agents in the presence of obstacle and connectivity constraints with mixed-integer linear programming. *International Journal of Robust and Nonlinear Control*, 30(14):5464–5491, 2020.

[15] Benicio Barbosa Cruz. Planejamento Ótimo de trajetórias para neutralização de ameaças aéreas com sistema multidrones e recursos limitados. Dissertação de mestrado, Instituto Tecnológico de Aeronáutica, São José dos Campos, SP, 2025.

[16] Bo Liu, Shulei Wang, Qinghua Li, Xinyang Zhao, Yunqing Pan, e Changhong Wang. Task assignment of UAV swarms based on deep reinforcement learning. *Drones*, 7(5):297, 2023. doi: 10.3390/drones7050297.

[17] Alessandro Palmas. Reinforcement learning for decision-level interception prioritization in drone swarm defense. *arXiv preprint arXiv:2508.00641*, 2025.

[18] Dianbo Jia, Ganliang Wang, Hongfei Bu, Junwei Wang, e Zhengkai Wang. Graph attention network-enhanced multi-agent reinforcement learning for dynamic interception task allocation in counter-drone defense. *Scientific Reports*, 2026. doi: 10.1038/s41598-026-55576-9.

[19] Laura Strickland, Michael A. Day, Kevin J. DeMarco, Eric Squires, e Charles Pippin. Responding to unmanned aerial swarm saturation attacks with autonomous counter-swarms. In *Proceedings of SPIE, Unmanned Systems Technology XX*, 2018.

[20] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

[21] James Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.

[22] Peter Swerling. Probability of detection for fluctuating targets. *IRE Transactions on Information Theory*, 6(2):269–308, 1960.

[23] Dimitri P. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14:105–123, 1988.

[24] Henk A. P. Blom e Yaakov Bar-Shalom. The interacting multiple model algorithm for systems with Markovian switching coefficients. *IEEE Transactions on Automatic Control*, 33(8):780–783, 1988.

[25] Zne-Jung Lee, Chou-Yuan Lee, e Shun-Feng Su. An immunity-based ant colony optimization algorithm for solving weapon-target assignment problem. *Applied Soft Computing*, 2(1): 39–47, 2002.

[26] Zne-Jung Lee, Shun-Feng Su, e Chou-Yuan Lee. Efficiently solving general weapon-target assignment problem by genetic algorithms with greedy eugenics. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 33(1):113–121, 2003.

[27] Brian P. Gerkey e Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004.

[28] Ravindra K. Ahuja, Arvind Kumar, Krishna C. Jha, e James B. Orlin. Exact and heuristic algorithms for the weapon-target assignment problem. *Operations Research*, 55(6):1136–1146, 2007.

[29] Han-Lim Choi, Luc Brunet, e Jonathan P. How. Consensus-based decentralized auctions for robust task allocation. *IEEE Transactions on Robotics*, 25(4):912–926, 2009. doi: 10.1109/TRO.2009.2022423.

[30] Stéphane Ross, Geoffrey Gordon, e Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.

[31] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, e Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[32] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, e Alexander Smola. Deep sets. In *Advances in Neural Information Processing Systems 30*, 2017.

[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, e Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

[34] Wouter Kool, Herke van Hoof, e Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.

[35] Fredrik Johansson. *Evaluating the Performance of TEWA Systems*. PhD thesis, Örebro University, 2010.

[36] N. A. Shneydor. *Missile Guidance and Pursuit: Kinematics, Dynamics and Control*. Horwood Publishing, 1998.

[37] Paul Zarchan. *Tactical and Strategic Missile Guidance*. American Institute of Aeronautics and Astronautics, 6 edition, 2012.

[38] Neil F. Palumbo, Ross A. Blauwkamp, e Justin M. Lloyd. Basic principles of homing guidance. *Johns Hopkins APL Technical Digest*, 29(1):25–41, 2010.

[39] Michal Pliska, Matouš Vrba, Tomáš Báča, e Martin Saska. Towards safe mid-air drone interception: Strategies for tracking & capture. *IEEE Robotics and Automation Letters*, 9 (10), 2024.

[40] Yaakov Bar-Shalom, X. Rong Li, e Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[41] Wild Hornets. Sting interceptor. https://wildhornets.com/en/sting-interceptor, s.d. Acesso em out. 2026.

[42] Unmanned Airspace. Wild hornets' sting interceptor reaches 315 km/h, shoots down over 200 shaheds and gerberas. https://www.unmannedairspace.info/?p=34271, 2025. Acesso em out. 2026.

[43] European Space Agency. Copernicus DEM – global and european digital elevation model (GLO-30). https://doi.org/10.5270/ESA-c5d3d65, 2021. Acesso em out. 2026.

[44] OpenStreetMap contributors. Openstreetmap. https://www.openstreetmap.org, 2026. Dados obtidos pela Overpass API em 15 out. 2026; licença ODbL.

[45] Diretoria Principal de Inteligência da Ucrânia (GUR). GERAN-2 (Shahed-136) – war & sanctions. https://war-sanctions.gur.gov.ua/en/uav/336, s.d. Acesso em out. 2026.

[46] GlobalSecurity.org. Geran-2. https://www.globalsecurity.org/military/world/russia/geran-2.html, s.d.. Acesso em out. 2026.

[47] Drone Warfare. Shahed-136 and the geran family: Variants, production, guidance and employment. https://drone-warfare.com/research/shahed-136/, s.d. Acesso em out. 2026.

[48] Institute for Science and International Security. Russian decoy drones that depend on western parts pose a great challenge to ukrainian defenses. https://isis-online.org/isis-reports/russian-decoy-drones-that-depend-on-western-parts-pose-a-great-challenge/, s.d. Acesso em out. 2026.

[49] Centre for Air Power Studies. Low-cost gerbera drones: The key player in russia–ukraine conflict. https://capssindia.org/?p=16521, 2025. Acesso em out. 2026.

[50] defence-network.com. Gerbera: Russlands günstige täuschungsdrohne. https://defence-network.com/gerbera-russland-taeuschungsdrohne/, s.d. Acesso em out. 2026.

[51] Grey Dynamics. Lancet 3: Russia's spear in the sky. https://greydynamics.com/lancet-3-russias-spear-in-the-sky, s.d. Acesso em out. 2026.

[52] GlobalSecurity.org. Lancet – specifications. https://www.globalsecurity.org/military/world/russia/lancet-specs.htm, s.d.. Acesso em out. 2026.

[53] Kyiv Post. 'can chase a helicopter' – ukrainian fpv drone reaches 325 km/h. https://kyivpost.com/post/39057, 2024. Acesso em out. 2026.